

32548-48730, Cybersecurity

WEEK 3

Public Key Infrastructure (PKI) and Certificate



Readings- Stallings – Chapter 3, Wenliang Du- Chapter 24

A/Prof. Priyadarsi Nanda
Priyadarsi.Nanda@uts.edu.au

Objectives



Cryptographic Functions

Symmetric vs Asymmetric encryption and hashing

Public Key Cryptography (PKC)

Secure key exchange over insecure channels, digital signatures

Digital Certificates

Trust via verifiable identities & Certificate Transparency

Certificate Authorities (CAs)

Who signs your key, why it matters, and public logging

Public Key Infrastructure (PKI)

Architecture behind global trust, automation via ACME, short-lived certs

X.509 Certificates

Standards used in SSL/TLS, digital identity, and post-quantum readiness

- How to find & analyze certificates from different sources?
- Why must a **public key be trusted**?
- What is the **chain of trust** in cryptographic systems?
- How do **CAs defend against MITM attacks**?
- What happens if a CA is compromised?



Why We Need Cryptographic Function in Cybersecurity?



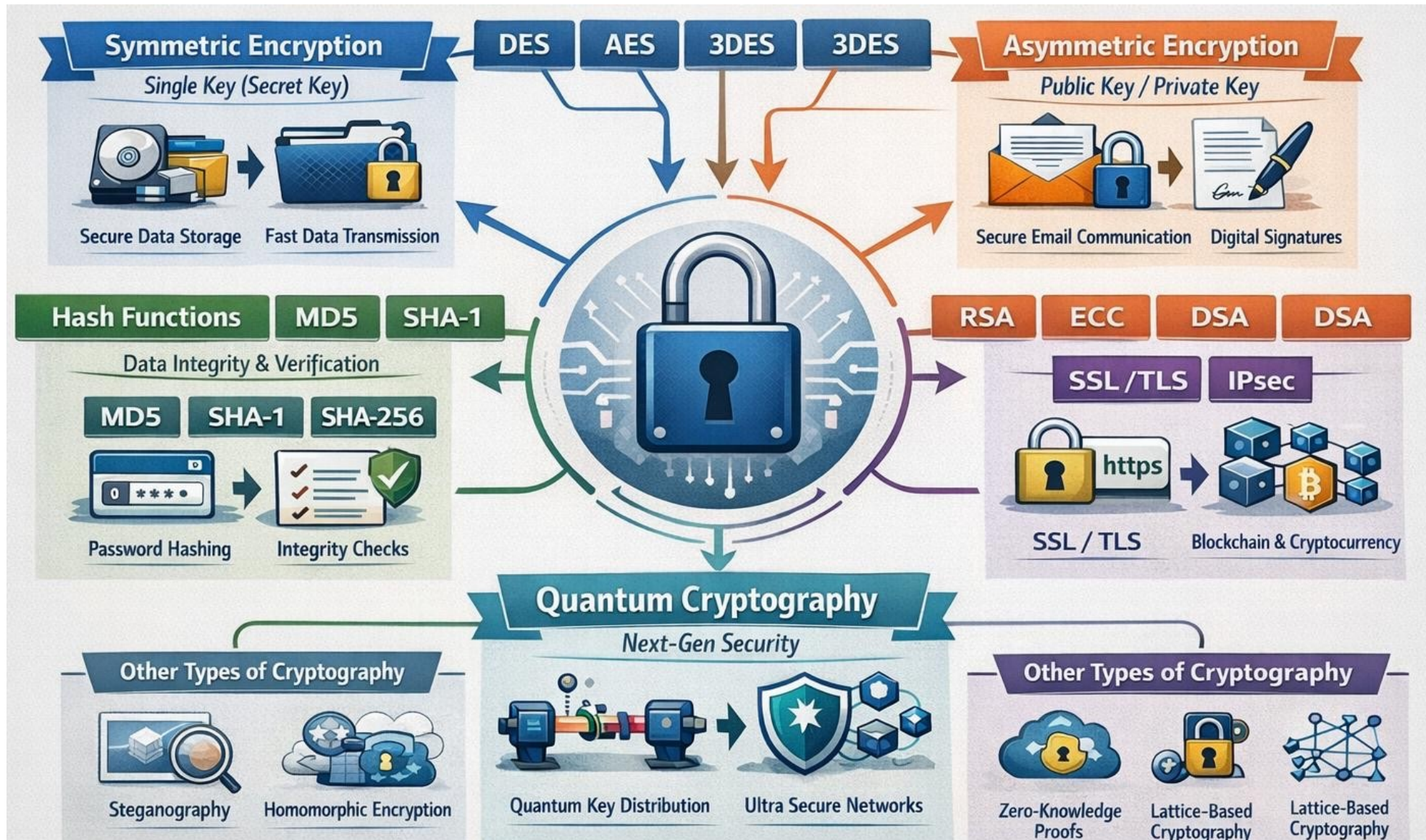
Cryptography powers digital trust, safeguarding communication, verifying identity, and ensuring integrity from your inbox to global cloud systems

Use of Cryptography in the Real World

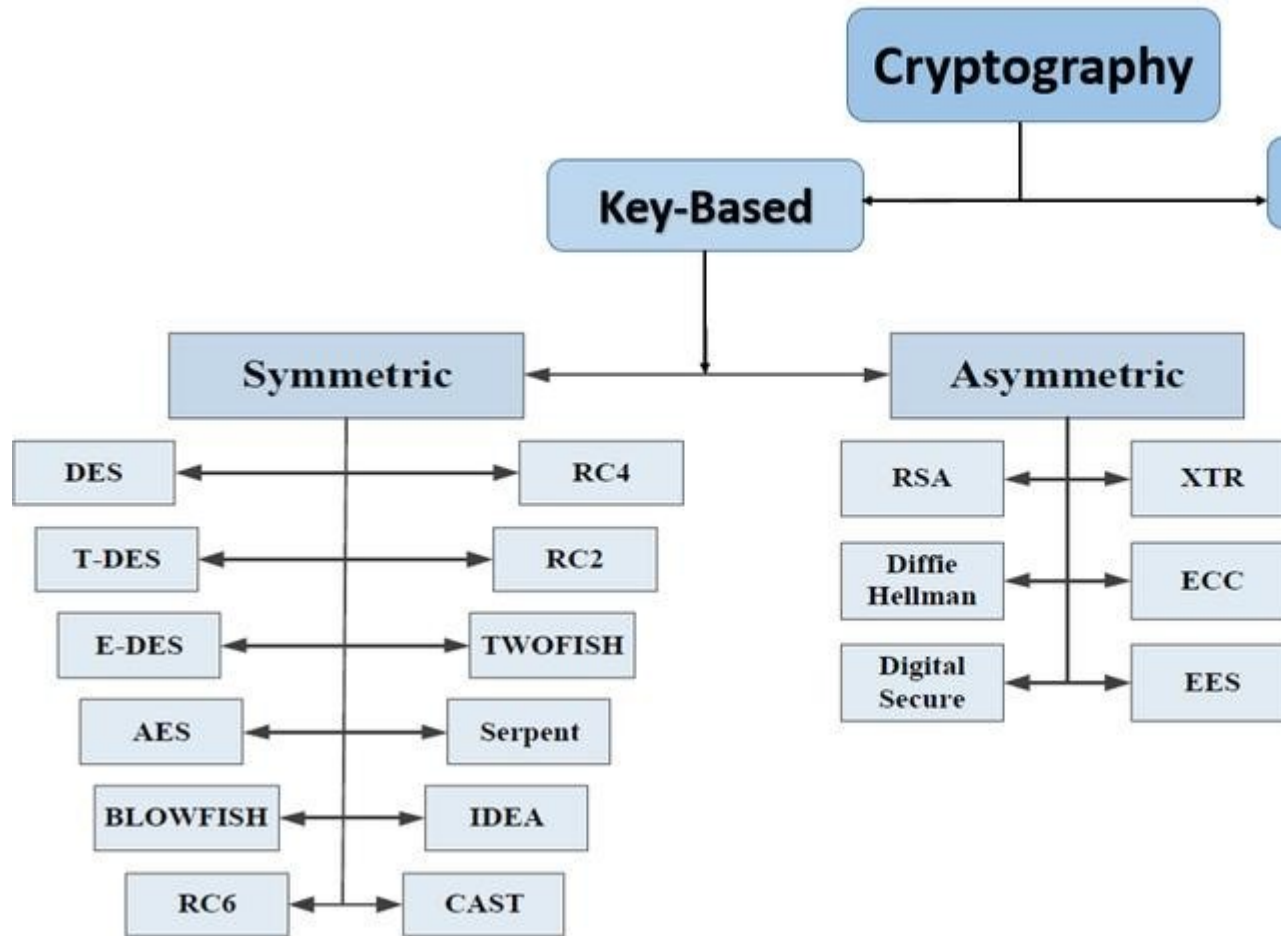


<u>Scenario</u>	<u>Crypto Used</u>	<u>Security Outcome</u>	<u>Threats Blocked</u>
Sending Gmail / Secure Messaging	🔑 TLS 1.3 + Asymmetric Encryption (RSA/ECDHE) + Perfect Forward Secrecy	Safe, private communication	Eavesdropping, Man-in-the-Middle (MitM), Downgrade Attacks
Logging into Bank	🔒 Authentication (MFA) + Secure Hashing (PBKDF2, Argon2)	Verified identity & secure login	Credential Theft, Credential Stuffing, Brute Force Attacks
Software Download / Update	✍️ Digital Signature (Ed25519, ECDSA)	Verified origin, untampered content	Tampering, Supply Chain Attacks, Fake Software Injection
Password Storage	🔑 Salted Hashing (bcrypt, scrypt)	Leak-resistant password protection	Password Dumps, Rainbow Table Attacks, GPU/ASIC Cracking
Cloud Data Backup	🔒 Client-side Encryption (AES-256, GCM mode)	End-to-end protection of cloud-stored data	Cloud Provider Data Breaches, Insider Threats
E-commerce Payment Processing	🔑 TLS 1.3 + Tokenization	Secure online transactions	Payment Data Theft, Replay Attacks

Types of Cryptography



Types of Cryptography



Caesar Cipher with a shift of 3

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

Shift: 0

Plain Text: Hello

Cipher Text:

Cryptographic Functions –Symmetric Encryption



Symmetric Encryption (Secret Key Cryptography) 🗝️

- *Single key* used for both **encryption** and **decryption**.
- *Key must be known* to both parties beforehand.
- **Fast** for large amounts of data.
- Requires **secure key exchange** method.
- **Use cases:** VPN traffic, database encryption, full-disk encryption.
- **Examples:** AES, ChaCha20, 3DES (*legacy*)

Symmetric encryption



Secret key



Reference documents:

AWS Key Management Service (AWS KMS) Overview – AWS Documentation

Symmetric vs Asymmetric Encryption – Cloudflare Learning Center

NIST Cryptographic Key Management Systems (CKMS)

Encryption Basics – OWASP Cheat Sheet Series

<https://docs.aws.amazon.com/kms/latest/developerguide/overview.html>

<https://www.cloudflare.com/learning/ssl/symmetric-vs-asymmetric-encryption/>

<https://csrc.nist.gov/projects/key-management/cryptographic-key-management-systems>

[https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic Storage Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html)

Cryptographic Functions – Asymmetric Encryption



Asymmetric Encryption (Public Key Cryptography) 🔑

- **Two keys:** Public key (*encrypt*) & Private key (*decrypt*).
- Easy **key distribution** (public key can be shared openly).
- Slower than symmetric methods for large data.
- **Use cases:** SSL/TLS certificates, secure email (PGP), digital signatures.
- **Examples:** RSA, ECC, Diffie–Hellman (*key exchange*)

Asymmetric encryption



Popular Algorithms & Key Sizes



Type	Algorithm	Key Size	Notes
🔑 Symmetric (Secret Key)	DES (legacy)	56-bit	Weak – easily brute-forced. Not recommended.
	AES (Advanced Encryption Standard)	128, 192, 256-bit	Modern standard, used in TLS, VPNs, disk encryption.
	IDEA (International Data Encryption Algorithm)	128-bit	Strong, less common than AES.
🔑 Asymmetric (Public/Private Key)	RSA	1024, 2048, 4096-bit	Widely used in SSL/TLS & email encryption. Use ≥ 2048 -bit.
	ECC (Elliptic Curve Cryptography)	160, 224, 256, 384, 521-bit	Strong security with shorter keys. Used in TLS, blockchain.
	Diffie–Hellman (DHKE)	1024, 2048, 4096-bit	Key exchange protocol. Use elliptic curve variant for better security.
	DSA (Digital Signature Algorithm)	1024, 2048, 3072-bit	Used for authentication & integrity.

Asymmetric Encryption- RSA



RSA (Rivest–Shamir–Adleman):

- **Named after its creators:** Rivest, Shamir, and Adleman.
- **Introduced in 1978** and still widely trusted today.
- **Relies on prime factorization** – the difficulty of factoring large numbers underpins its security.
- **Used in secure communications** (SSL/TLS), digital signatures, and cryptographic protocols.
- **Security strength:** Resistant to classical computing attacks but, potentially vulnerable to future quantum computing advancements.



Using **2048-bit RSA keys** is currently the minimum requirement for strong security; **3072-bit+** recommended for long-term protection.

RSA- Generating Public and Private Keys



Step 1: Pick two prime numbers

❑ Example: $p = 5, q = 13$

Step 2: Calculate n

❑ $n = p \times q = 5 \times 13 = 65$
▪ Used in both public & private keys.

Step 3: Calculate z (Totient)

❑ $z = (p - 1) \times (q - 1) = 4 \times 12 = 48$

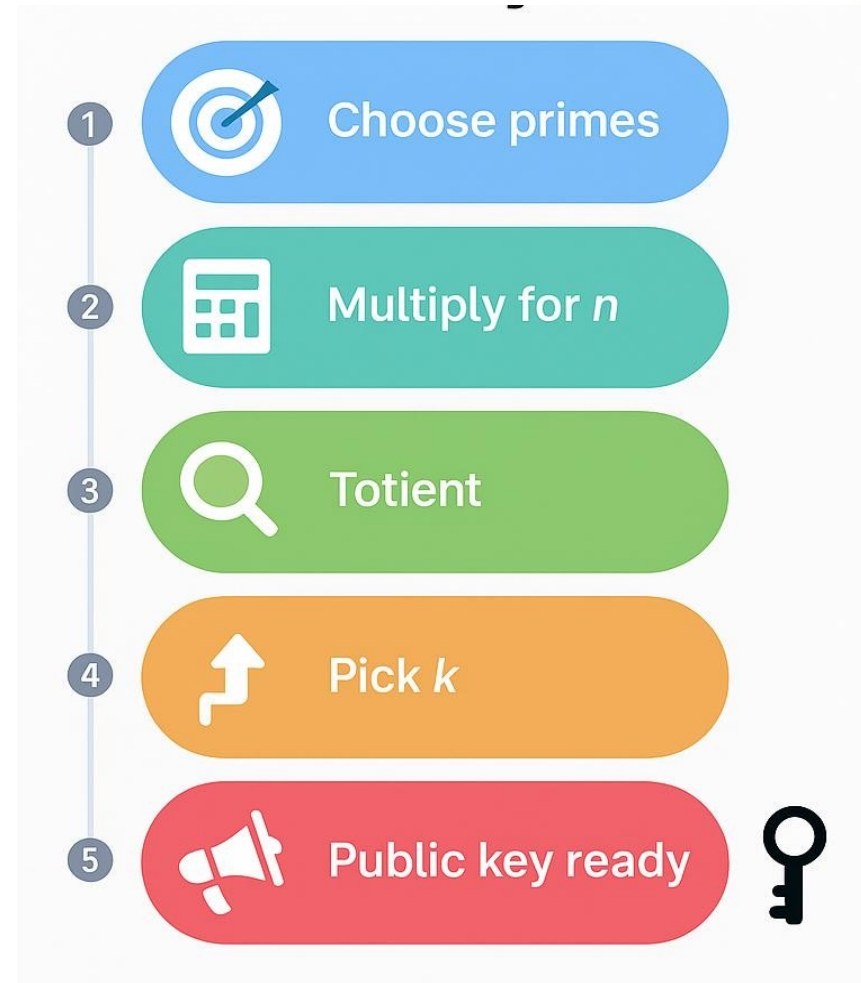
❑ **Step 4: Pick a public exponent k (must be co-prime to z)**

❑ Possible: 7, 11, 17, 19, 23

❑ Choose $k = 11$

❑ **Step 5: Form the Public Key**

❑ **Public Key = $(n = 65, k = 11)$**



RSA- Generating Public and Private Keys



Step 1:

We know:

□ $k \times j \equiv 1 \pmod{z}$

□ $k = 11, z = 48$

Step 2:

➤ Substitute:

□ $11 \times j \equiv 1 \pmod{48}$

Step 3:

Find j :

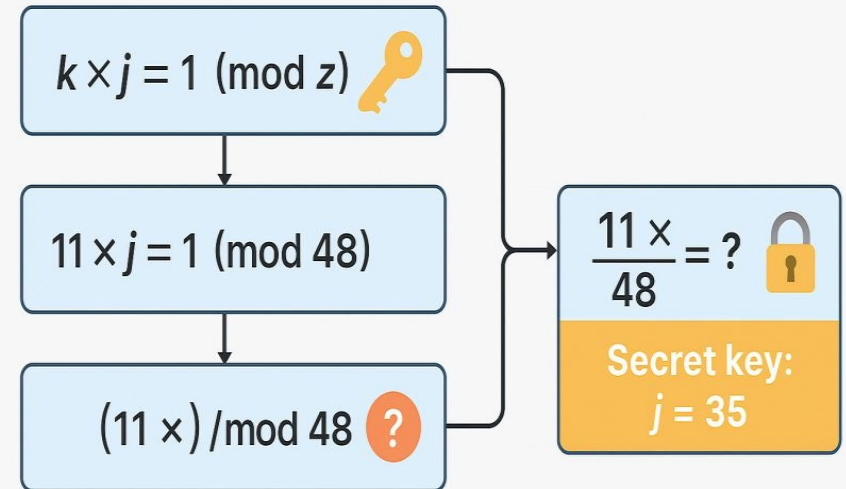
□ Multiply 11 by 35 $\rightarrow 11 \times 35 = 385$

□ $385 \div 48 = 8 \text{ remainder } 1$ ✓

Step 4:

□ **Secret key** $\rightarrow j = 35$

RSA: Calculating the Secret Key



Key takeaway:

The secret key j is the modular multiplicative inverse of k modulo z .

Message encryption using RSA



- Here's the encryption math the browser executes:

$$P^k \equiv E \pmod{n}$$

Variables:

- $P \rightarrow$ Plain message to encrypt
- n and $k \rightarrow$ Server's public key components
- $E \rightarrow$ Encrypted message generated

Example:

- Given: $P = 14, n = 65, k = 11$
- Substitute into the formula:

$$14^{11} \equiv E \pmod{65}$$

- Raise 14 to the power of 11, then divide by 65 and take the remainder $\rightarrow$

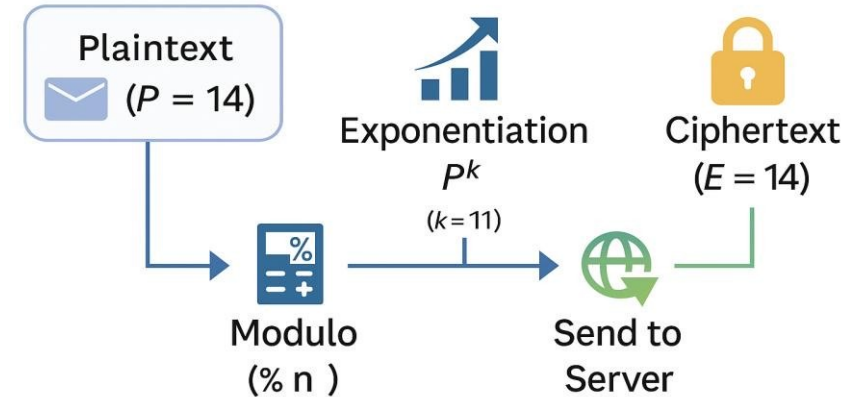
Using modular exponentiation:

$$14^{11} \bmod 65 = 14$$

- Result: $E = 14$ (Transmitted value)

Transmission:

- ✓ This encrypted value **E** is what the browser sends to the server.
- ✓ Without the private key, reversing the operation is computationally infeasible for large n .



Decrypting the Message RSA Example



- Here is the decryption math the server uses to recover the original plain text message that the browser sent.

$$E^j = P \bmod n$$

- $E \rightarrow$ Encrypted message just received.
- $j \rightarrow$ Server's secret key.
- $P \rightarrow$ Plain message to recover.
- $n \rightarrow$ Server's public key.

Step-by-Step:

1. Plug in values:

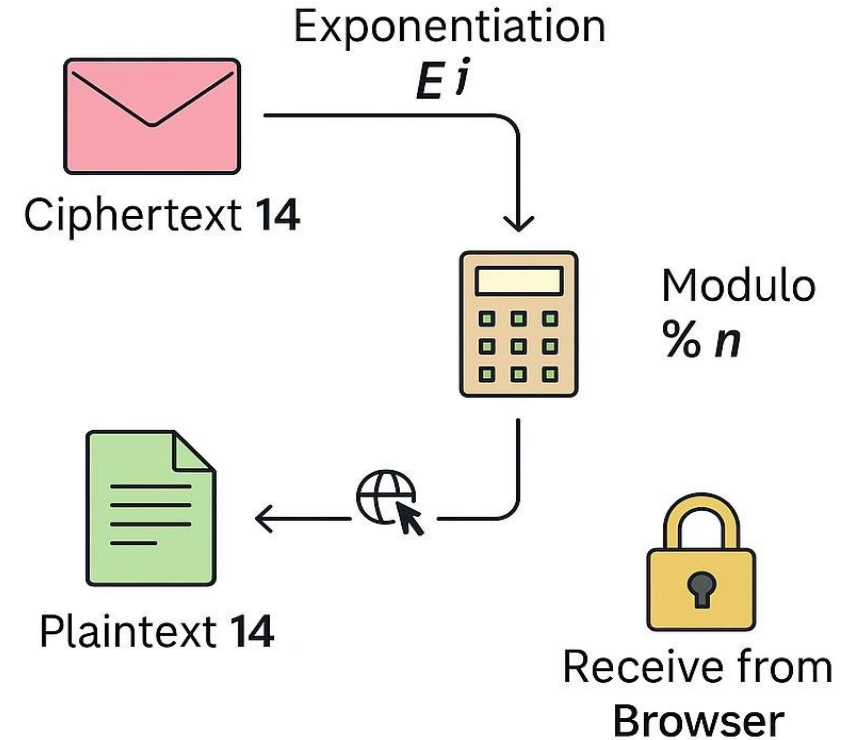
$$14^{35} = P \bmod 65$$

2. Perform modular exponentiation:

$$14^{35} \bmod 65 = 14$$

3. Decrypted Message:

$P = 14$ — exactly the original plain text.



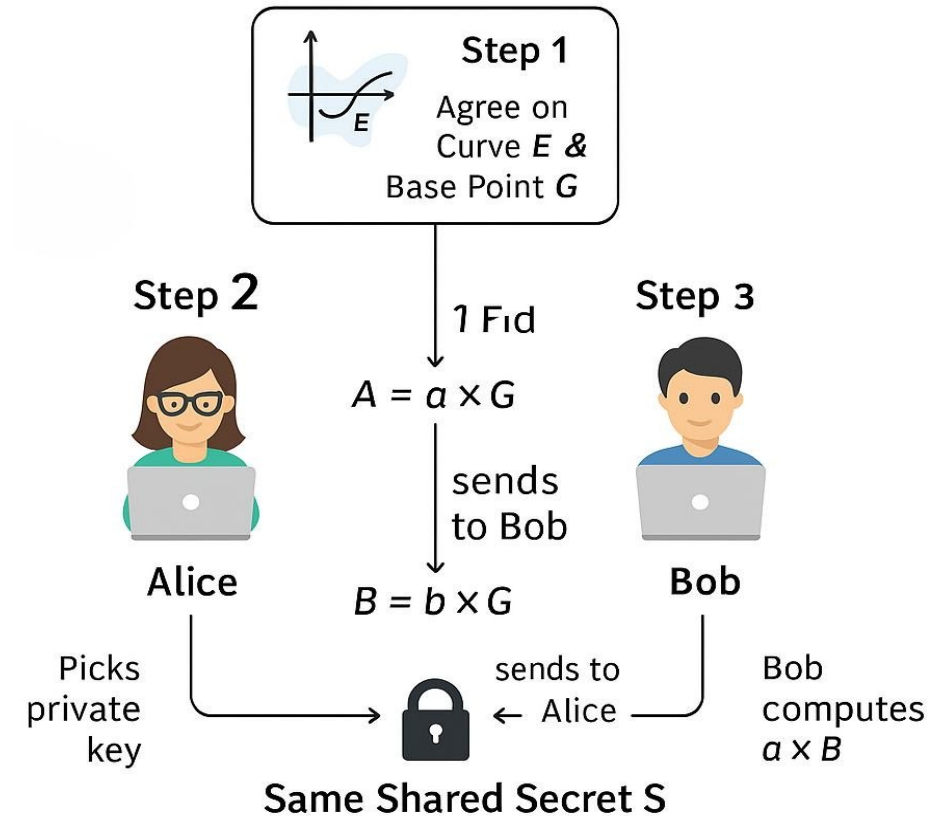
Elliptic Curve Cryptography (ECC) Modern Asymmetric Encryption



- ECC is a form of public-key cryptography based on the algebra of elliptic curves over finite fields.
- **Why ECC?**
 - Smaller keys → same security strength as RSA.
 - Faster computations → good for mobile and IoT devices.
 - Lower bandwidth usage.
- **Security Basis:**
 - Relies on the *Elliptic Curve Discrete Logarithm Problem (ECDLP)* – extremely hard to solve.

Example (Simplified ECC Key Exchange)

1. **Alice's Side:** Picks private key a , computes public key $A = a \times G$.
2. **Bob's Side:** Picks private key b , computes public key $B = b \times G$.
3. **Exchange Public Keys:**
 - Alice receives B , computes $S = a \times B$.
 - Bob receives A , computes $S = b \times A$.
4. **Shared Secret:** Both get the same point S — can be used for symmetric encryption.
5. **Public Curve & Base Point:** Both parties agree on curve E and base point G .



Hybrid Encryption – RSA + AES (SSL/TLS Style)

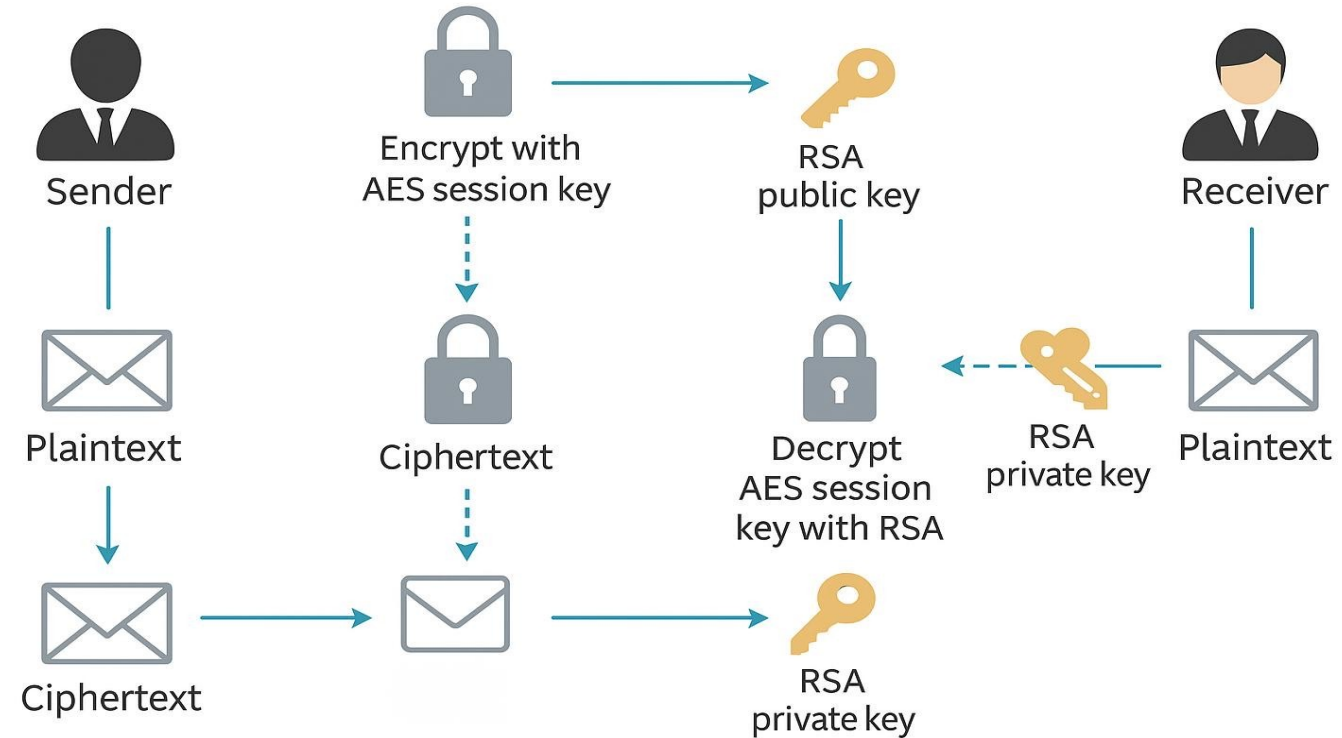


Hybrid encryption combines **asymmetric encryption** (RSA) with **symmetric encryption** (AES) for efficiency and security.

- ❑ **RSA** secures the exchange of the **AES session key**.
- ❑ **AES** encrypts the actual data — faster and more efficient for large amounts of data.
- Used in **SSL/TLS** to establish secure HTTPS connections.

Benefits:

- Speed of AES
- Forward secrecy (if ephemeral keys are used)
- Security of RSA

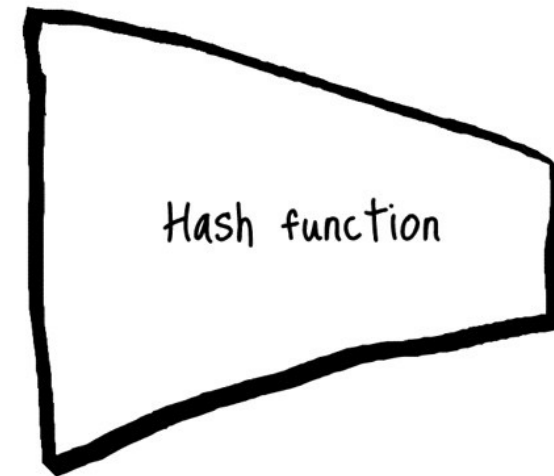


- **Sender generates** a random AES session key.
- **Sender encrypts** the AES key with **Receiver's RSA public key** → sends it to the server.
- **Receiver decrypts** the AES key with its **RSA private key**.
- Both now share the same AES key.
- All further communication is encrypted with AES.

Hash Function



- Hash is a **one-way function**. In other words, a one-way hash can be generated from any piece of data, but the data cannot be generated from the hash.
- A hash function returns message digest or hash values.
- Its features:
 - **Fixed length output:** popular hash functions generate values from 128 to 512 bits.
 - **Efficiency of operation:** computationally hash functions are much **faster than a symmetric encryption**



Public Key Cryptography-concept



Further reading on public-key encryption

Source & Link

Cloudflare – How does public key cryptography work? cloudflare.com

GeeksforGeeks – Public Key Encryption geeksforgeeks.org

MetaSchool – Public Key Cryptography Explained metaschool.so

Howik – Understanding Public Key Cryptography howik.com

History Tools – The Complete Guide to Public Key Cryptography historytools.org

PreVeil – Public & Private Key: How they Work Together preveil.com

MDN Glossary – Public-key cryptography developer.mozilla.org

Description

A reliable, modern overview focusing on TLS/SSL, HTTPS, and asymmetric encryption in secure web communications.

Concise technical explanation with examples of encryption/decryption using public and private keys.

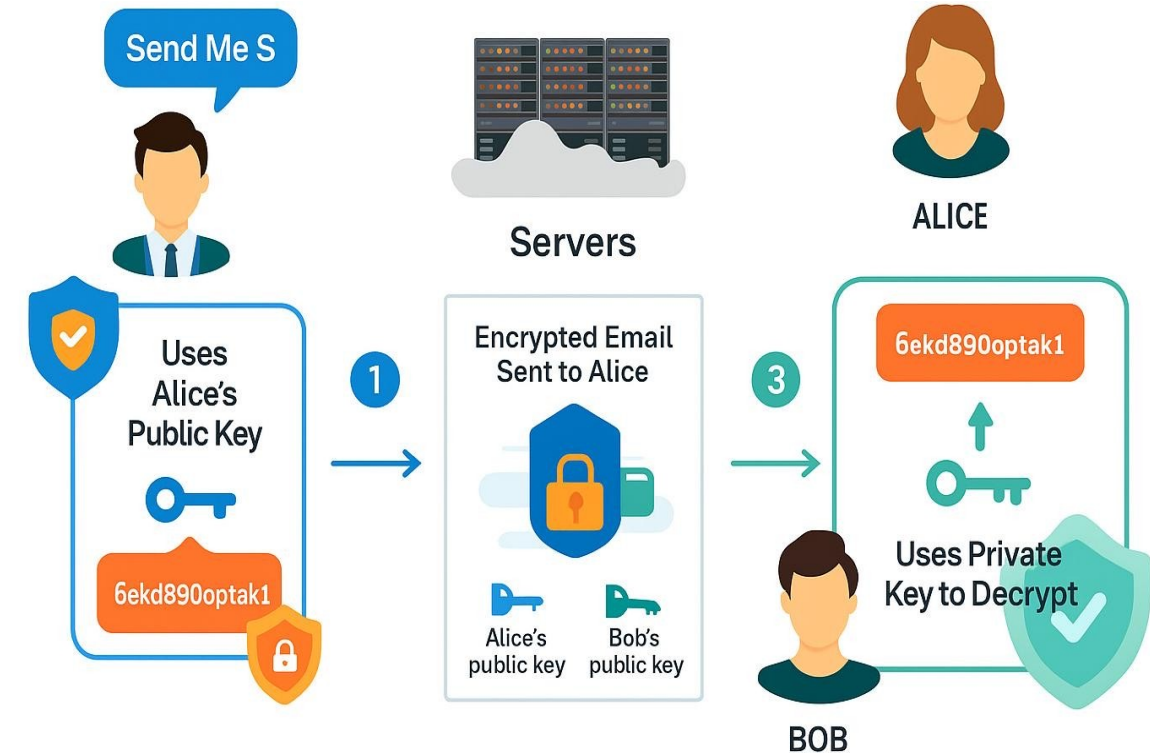
Detailed 2025 guide covering history, RSA, Diffie–Hellman, ECC, PKI, signatures, and future trends.

Beginner-friendly guide using analogies and real-world applications.

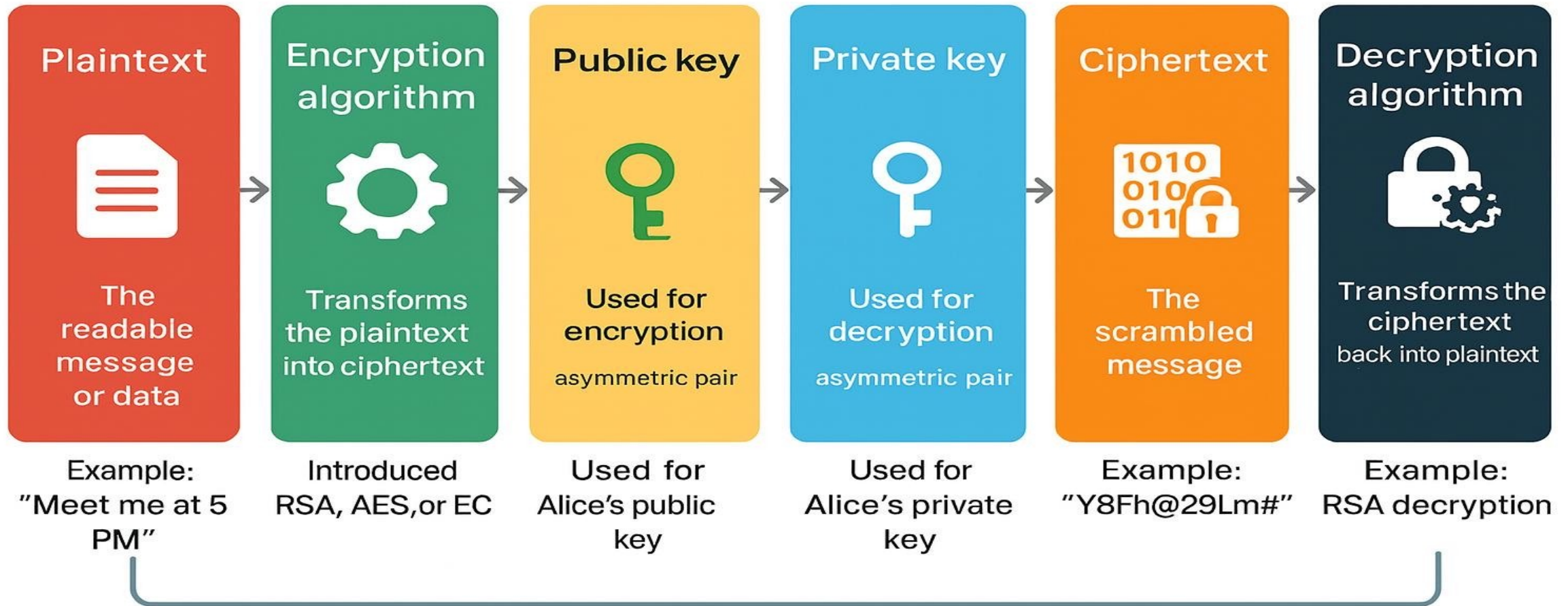
Practical step-by-step walkthrough with Alice & Bob examples for better conceptual understanding.

Practical explanation comparing public keys to addresses and private keys to door keys.

Authoritative reference from Mozilla covering asymmetric encryption basics and common algorithms.

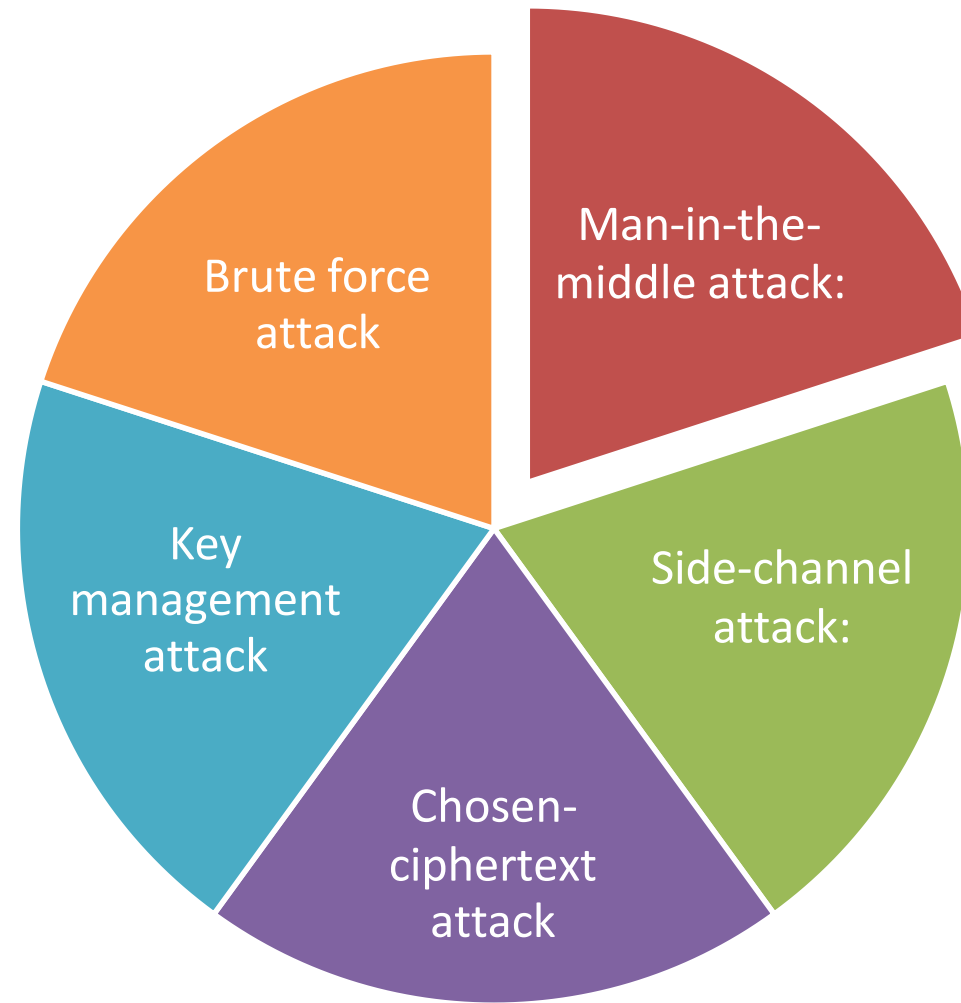


Public-Key Cryptography- 6 Ingredients



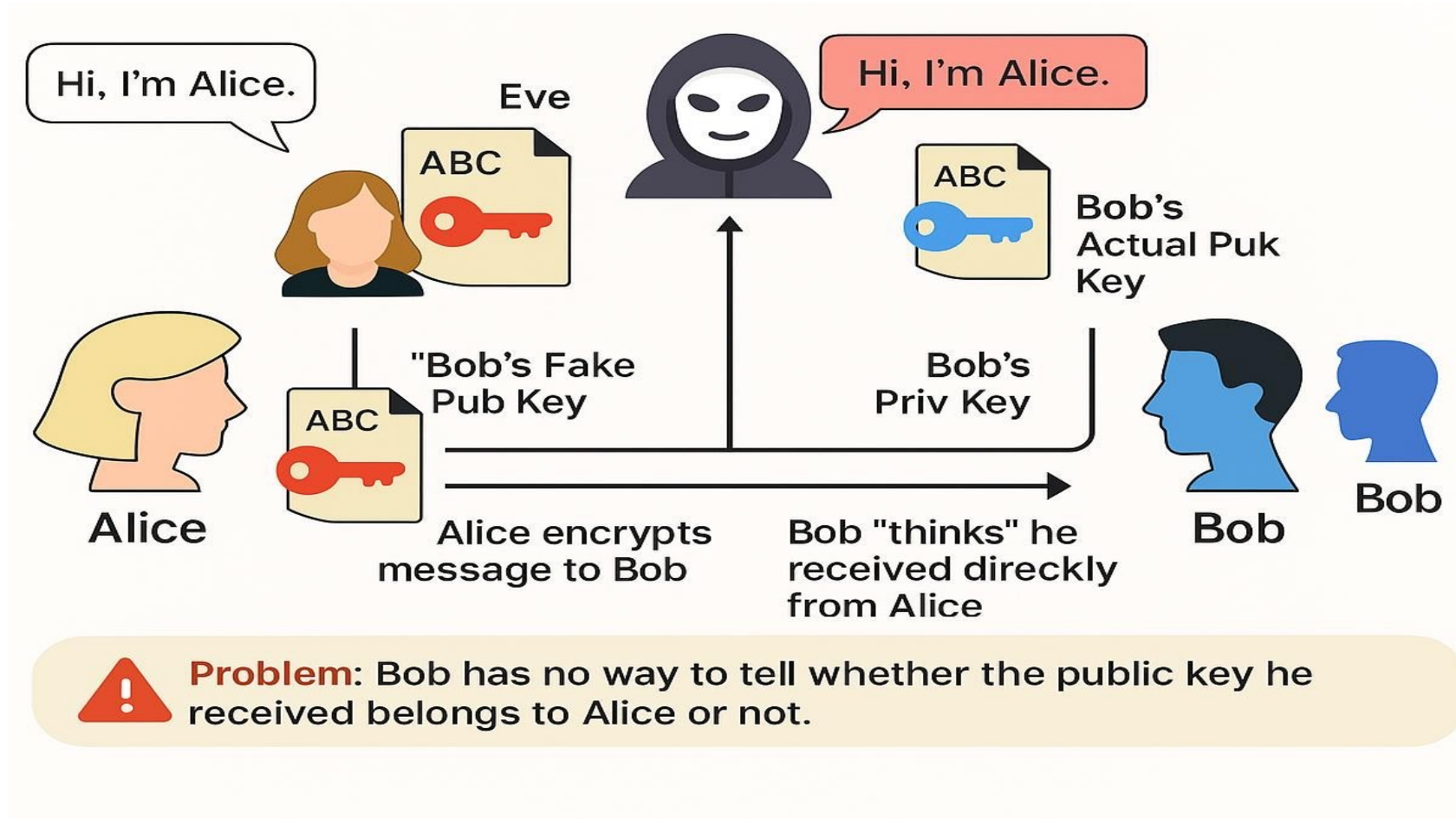
"These six ingredients are the building blocks of secure communication over the internet."

Vulnerabilities-Public Key Cryptography



https://www.researchgate.net/publication/374275275_Cyber-attacks_on_Public_Key_Cryptography

Man-in-the-Middle (MITM) Attack



MITM Attacks-Solution using Digital Signature



Solution:

Use a **Certificate Authority (CA)** to bind identity to a public key.

- Find a **trusted third party** — verifies the claimed identity.
- Bind the **identity** to a public key in a **digitally signed certificate**.
- Certificate **cannot be forged or altered** due to the CA's digital signature.

Steps:

- Alice requests a certificate from the CA.
- CA verifies Alice's identity and issues a signed certificate with her name + public key.
- Alice sends the full certificate to Bob.
- Bob verifies it using the CA's public key.
- Bob now knows the **true owner** of the public key

Result: Public keys are trusted and MITM attacks via fake keys can be blocked.

Trusted key ownership

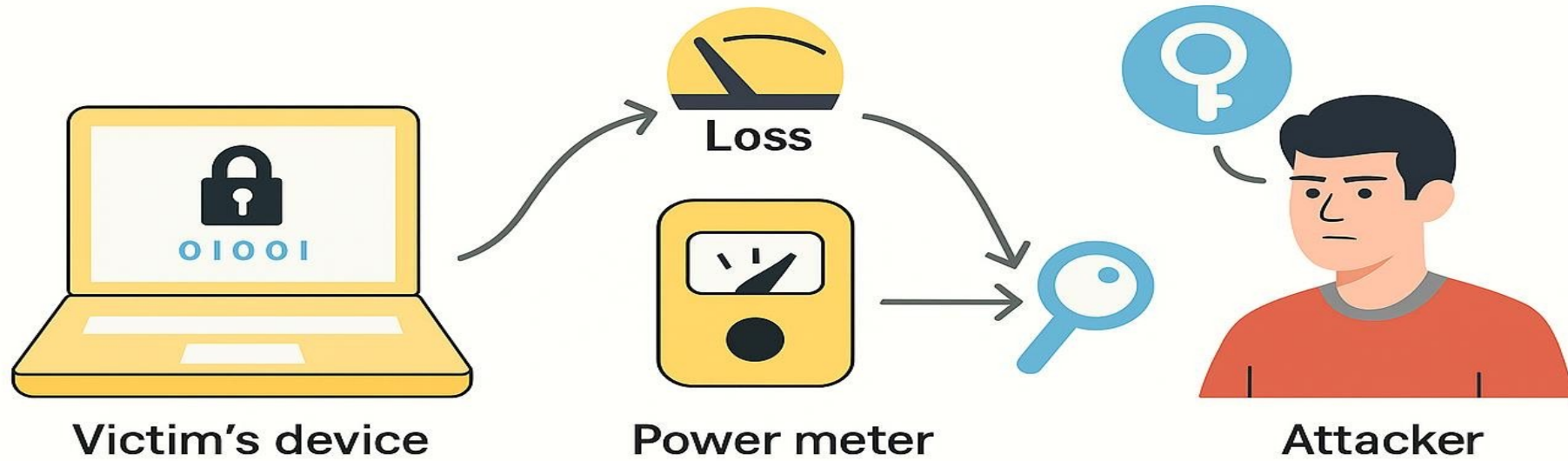
Secure communication
established

Untrusted public keys

Vulnerable to key forgery



Side-Channel Attack



Problem: Simple passwords are more prone to being brute-forced.

Side-Channel Attack: Solution using Constant-Time + Blinding



Solution:

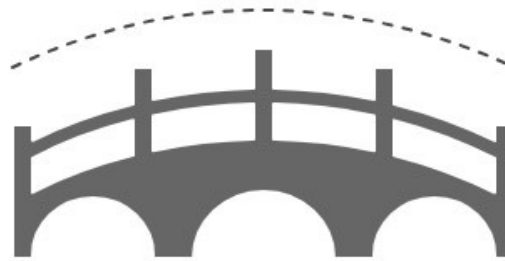
Constant-time code & noise to hide secrets

- 🕒 **Use constant-time implementations** — no data-dependent branches or table lookups.
- 🎲 **Apply masking/blinding** — randomize intermediate values during crypto ops.
- 🏠 **Reduce leakage** — shield hardware, add power/EM noise, and sample at fixed clock rates.



Vulnerable Crypto Operations

Secret keys are
susceptible to side-
channel attacks.



Secure Crypto Operations

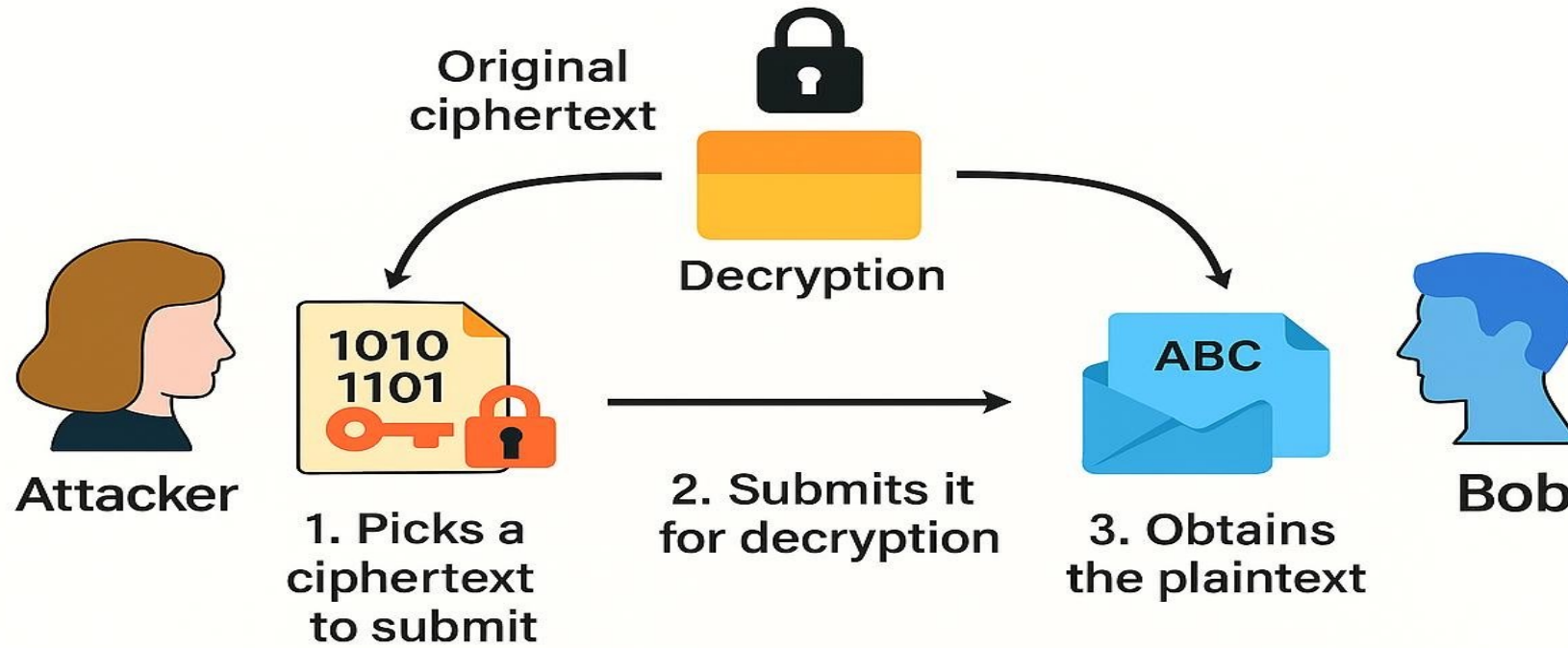
Attacker
measurements no
longer correlate with
the secret key.

Steps:

- ✅ Replace vulnerable code with constant-time libs (AES, RSA/ECC).
- 🗝️ Add **RSA blinding** / **ECDSA nonce randomization**.
- 🏠 Test with side-channel tools; set alarms for unusual power/EM patterns.

Result: Attacker measurements (power/EM/timing) no longer correlate with the secret key.

Chosen-Ciphertext Attack



Problem: Decryption oracle allows attacker to gradually learn sensitive data.

Chosen-Ciphertext Attack (CCA): Solution using AEAD/OAEP



Solution:

Use CCA-secure schemes & hide errors

■ Prefer **AEAD** (AES-GCM or ChaCha20-Poly1305) — encrypts **and** authenticates.

🔒 If RSA is needed, use **RSA-OAEP** (never PKCS#1 v1.5).

🚫 **No padding-oracle clues** — return a single generic error; constant-time checks.

Steps:

✅ Decrypt **only after** authentication passes (verify-then-decrypt).

🕒 Make failure paths indistinguishable (time, text, and codes).

■ Add rate-limits & logging on decryption endpoints.

Result: Crafted ciphertexts reveal nothing; oracles are shut down.



Key Management Attack: Solution using HSMs & Rotation

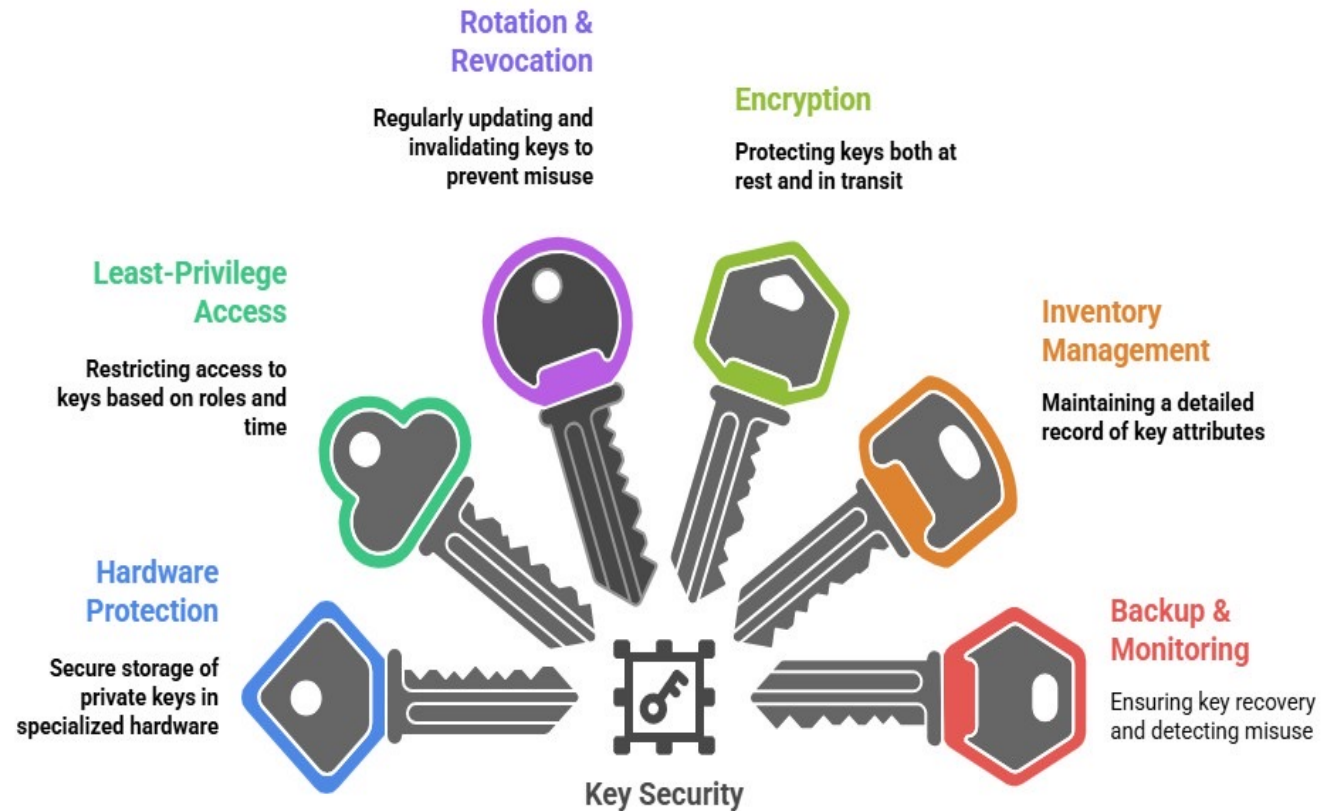


- Poorly stored keys may be subjected to attack

Solution:

Protect the full key lifecycle

- **Hardware protection** — store private keys in HSM/TPM/secure enclave.
- **Least-privilege access** — RBAC, MFA, short-lived credentials, audited access.
- ♻️ **Rotation & revocation** — expire, rotate, and revoke keys on schedule/events.



Steps:

- 🔒 Encrypt keys at rest & in transit; separate duties (split knowledge/M-of-N).
- 📋 Keep an inventory: owner, purpose, algorithm, length, expiry.
- Continuous backups & recovery tests; monitor for key misuse.

Result: Even if a server is compromised, keys remain protected or quickly unusable.

Brute-Force Attack: Solution using Strong Keys & Throttling



- **Attackers use automated tools with powerful machines to guess passwords trying multiple possibilities**

Solution:

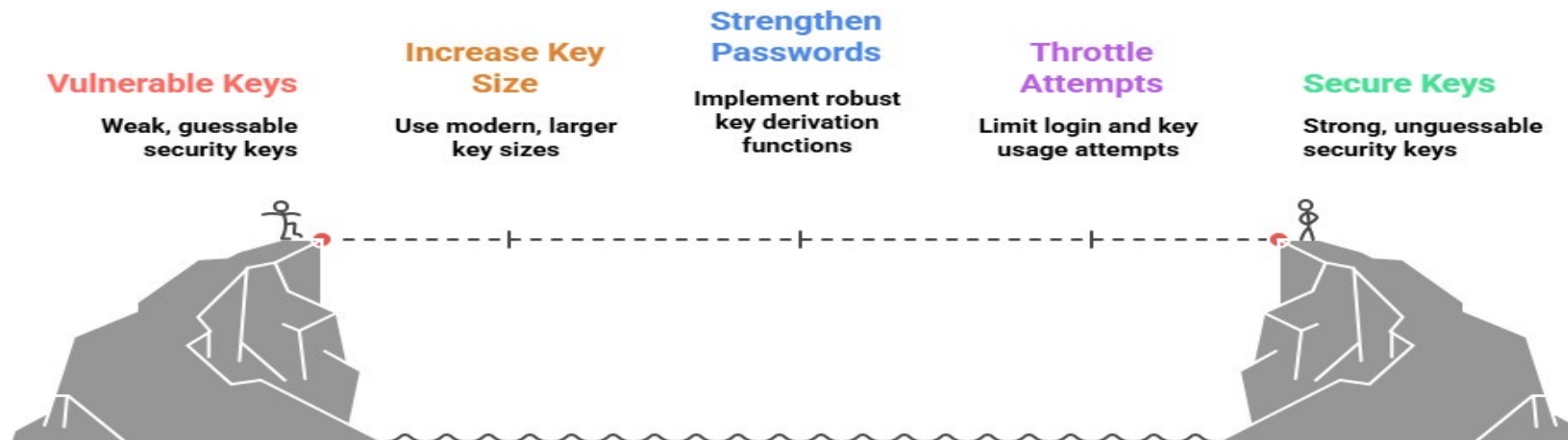
Increase work factor and limit guesses

- 🔑 **Use modern key sizes** — RSA ≥ 2048 -bit, ECC ≥ 256 -bit, AES-256.
- **For passwords/derived keys** — PBKDF2/bcrypt/scrypt/Argon2 with high cost.
- 🛑 **Throttle attempts** — rate-limits, exponential back-off, lockouts, CAPTCHA, MFA.

Steps:

- ✅ Enforce strong randomness (CSPRNG) for keys and nonces.
- 💡 Add **2FA/MFA** for logins and key usage.
- 📈 Monitor and alert on high-volume failures or unusual IP patterns.

Result: Search space becomes astronomically large; online guessing becomes impractical.



Key Management Tools Benefit



Tool	OS Support	Encrypts	Best For	Link
Kleopatra	Windows, Linux (via Gpg4win or KDE)	Files, Emails	GUI for managing OpenPGP and X.509 certificates, encrypting/signing data	https://www.gpg4win.org/kleopatra.html
GnuPG	Windows, macOS, Linux	Files, Emails, Streams	Command-line based OpenPGP for advanced users across platforms	https://gnupg.org/
Mailvelope	Web Browsers (Firefox, Chrome)	Emails (webmail interfaces)	OpenPGP email encryption in browser	https://www.mailvelope.com/
AES Crypt	Windows, macOS, Linux, Mobile	Files	Simple AES file encryption across platforms	https://www.aescrypt.com/
AxCrypt	Windows, macOS, Mobile	Files	User-friendly, shell-integrated file encryption	https://www.axcrypt.net/
Encrypto	Windows, macOS	Files	Quick drag-and-drop file encryption solution	https://macpaw.com/encrypto
GPG Suite	macOS	Emails, Files	Seamless OpenPGP integration into Apple Mail and services	https://gpgtools.org/
WinGPG	Windows	Files, Key Management	Lightweight tray-based OpenPGP key manager	https://www.scar5.com/wingpg

How Public Key Infrastructure (PKI) Works?



PKI is the framework used to set up and manage **public key encryption**. It includes software, hardware, policies, and processes for creating, distributing, managing, storing, and revoking digital certificates.



PKI ensures secure communications, authentication, and data integrity.

PKI Components



❑ **Certificate Authority (CA)**

- Trusted third party that verifies identity.
- Binds identity to a public key.
- Issues digitally signed certificates.
- Maintains a Certificate Revocation List (CRL).

❑ **Digital Certificates**

- Bind a public key to an entity's identity.
- Include owner, organization, public key, serial number, issuing CA.
- Enable authentication, encryption, and signatures.
- Follow the X.509 standard.

❑ **X.509 Standard**

- Defines format, fields, and validation rules.
- Ensures cross-platform compatibility.
- Supports certificate chaining and key usage policies.

PKI Components



Certificate Authority

- Trusted third party verifies the claimed identity



Digital Certificates

- Documents that bind a public key to an entity's identity.

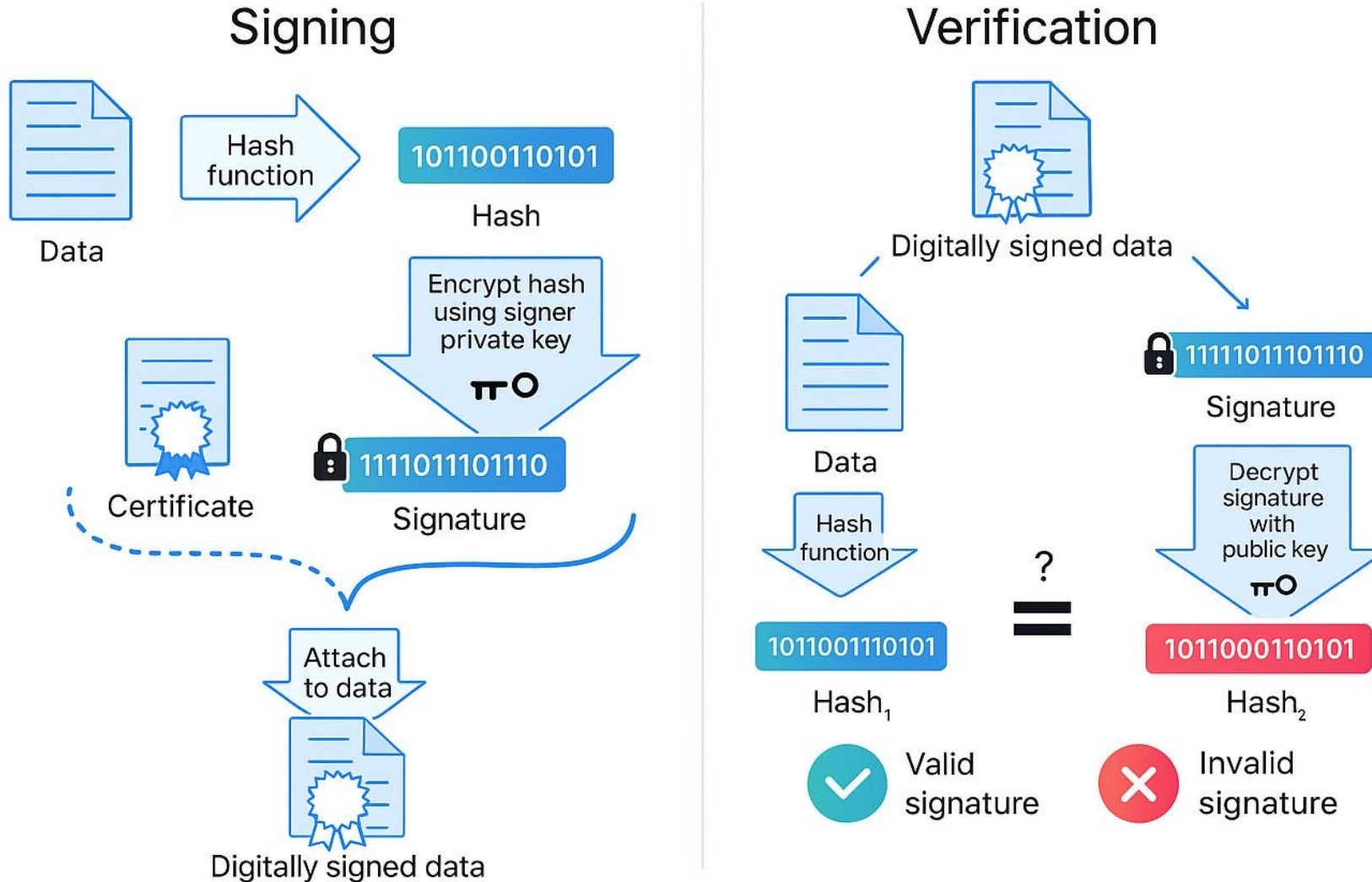


X.509 Standard

Defines format of public key certificates

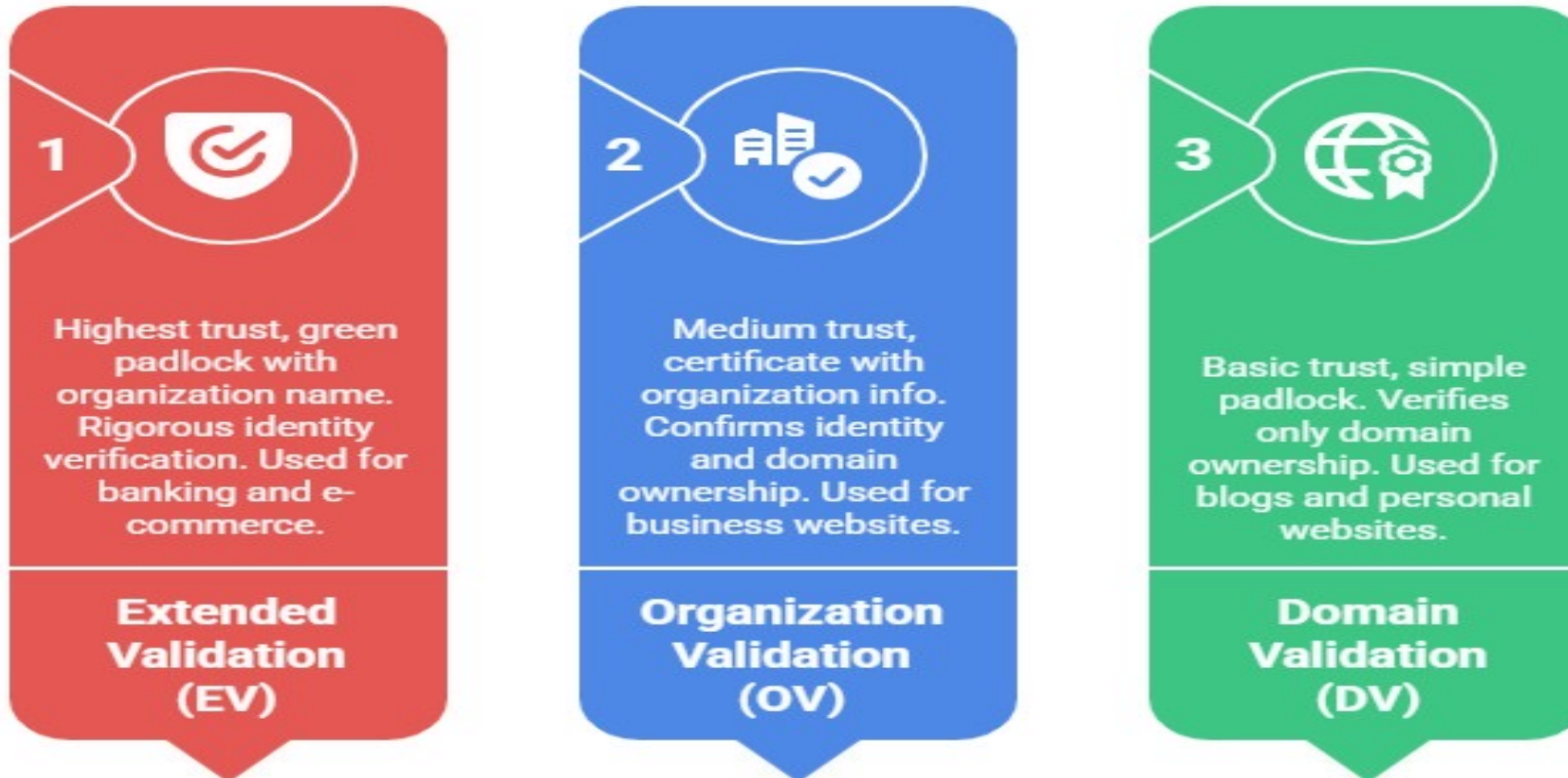
<https://engineering.purdue.edu/kak/compsec/NewLectures/Lecture13.pdf>

How Digital Signature/Certificate Works?



<https://proprivacy.com/email/guides/digital-signature>

Types of Digital Certificate



Type	Validation Level	Trust Level	Best For
EV 🛡️	Full legal validation	Highest	Banks, e-commerce, large enterprises
OV 🏢	Domain + Organization	Medium–High	Business websites
DV 🌐	Domain only	Low–Medium	Blogs, personal sites

Extended Validated Certificates (EV)



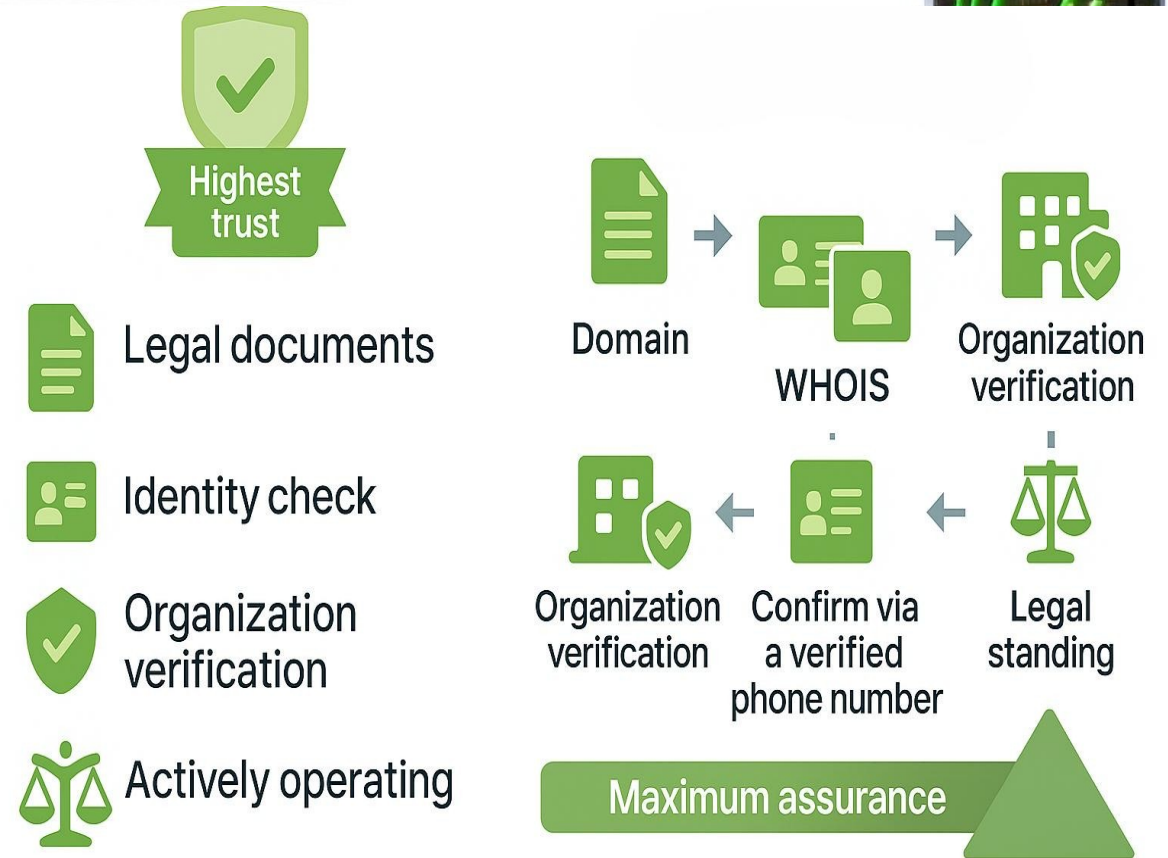
How EV Verification Works –

- Legal Docs: Signed from official registration authorities.
- Identity Check: Confirms individual's authority, signature, and link to the organization.

Organization Verification: Confirms physical address and telephone number.

Operational Existence: Ensures the company is actively operating.

Legal Standing: Confirms good legal status of the organization.



✓ **Features** – Maximum assurance, displays organization name in browser address bar, strong user confidence, most secure for high-value transactions.

⚠ **Limitations** – Higher cost, longer issuance time (days to weeks), requires ongoing compliance.

📌 **Use Cases** – Banks, large e-commerce platforms, government portals, enterprises handling sensitive or financial data.

Organizational Validated Certificates (OV)



Medium-trust SSL/TLS; verifies both domain ownership and the organization's identity.

How OV Verification Works –

 Domain Control Validation (DCV): Confirms the applicant controls the domain.

■ Identity & Address Check: Verifies the legal name, address, and contact details.

 Organization Link: Confirms applicant's association with the organization.

 Physical Address: Verified against public/business records.

 WHOIS Record Review: Matches registration details.

 Phone Callback: Confirms via organization's verified number.

✓ **Features** – Shows organization name in certificate details, better trust than DV, suitable for business transactions.

⚠ **Limitations** – Not as rigorous as EV; no green address bar display.

 **Use Cases** – Medium-sized businesses, NGOs, and organizations wanting extra credibility without EV cost/time.



Domain Validated Certificates (DV)



Most common SSL/TLS type; verifies only domain ownership without checking organization identity.

How DV Verification Works (DCV) –

 **Email:** Confirmation to admin WHOIS contact or standard addresses.

 **HTTP/HTTPS:** Upload CA-provided file/token to website directory.

 **DNS:** Add TXT record with CA-provided token.



Prove domain ownership



Email verification

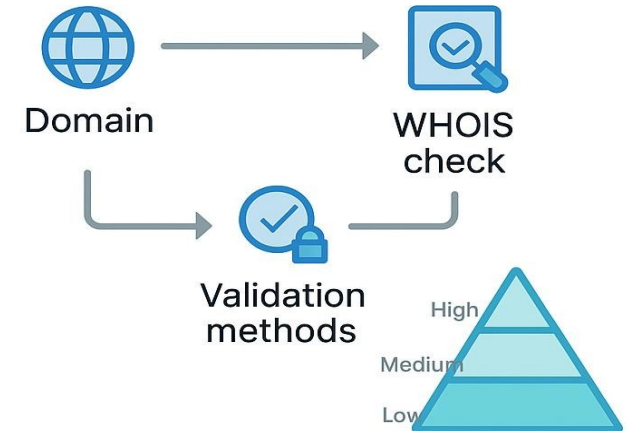


WHOIS check



Issued quickly

Domain Control Validation (DCV)



- ✓ **Features** – Quick (minutes), low-cost or free (e.g., Let's Encrypt), encrypts traffic, padlock in browser.
- ⚠ **Limitations** – No identity assurance, phishing risk for lookalike domains, unsuitable for high-trust sites.
- 📌 **Use Cases** – Blogs, personal sites, small business pages, dev/test servers.

Digital Certificate-Methodology



- Creating Digital Certificate

1 Step 1

```
$ openssl s_client -showcerts  
-connect www.paypal.com:443  
</dev/null
```



Connects to the server and retrieves its certificate chain.

2 Step 2



```
--BEGIN CERTIFICATE--  
BMEGFPIPAIBHJGXNHJZEVFF  
ZGJHQBZRtSQBPDLCH3RVrtz  
CJQU .....  
--END CERTIFICATE--
```

Encoded X.509 certificate in PEM format

3 Step 3

```
$ openssl x509  
-in paypal.pem -text -noout
```



Preview:

- Issuer
- Subject
- Expiry date

✓ The outcome of above command will show the certificate information

Decodes the PEM file to display human-readable certificate info

Example of X.509 Certificate (1st Part)



- **CA's Identity (Issuer)** → 🏢 “Who issued this certificate”
- **Validity Dates** → 📅 “Active period of the certificate”
- **Owner (Subject)** → 🌐 “Who owns this certificate”

- ☐ “Issuer: Trusted authority that signed the certificate.”
- ☐ “Subject: Entity to which the certificate is issued.”
- ☐ “Validity: Period during which the certificate is trusted.”

✓ **Tip: Always check the Issuer and Subject before trusting a certificate.**


CA's Identity
(Issuer)


Certificate
Owner
(Subject)

Certificate:

Version: 3

Serial Number:

2C:D1:95:10:54:37:D0:DE:4:39: 26:05:6a,F6/C2:7F

Signature Algorithm: sha256WithRSAEncryption

Issuer: C=US, O=Symantec Corporation, OU=Symantec

Trust Network, # CA that signed this cert

Validity

Not Before: Feb 2 00:00:00 2024 GMT

Not After : Oct 30 23:59:59 2025 # Expiry time (UTC)



Validute

C=🟢, ST=Califolifirania, L=San Jose, street=2211 N 1st St,

O=🟢ayPal, Inc., OU=CDN Support, CN=www.paypal.com

businessCategory=Private Organization,

serialNumber=3014267 # Owner

Subject Public Key Info:

Public Key Algorithm:rsaEncryption

RSA 2048, e=65537



Signature Algorithm



High
Trust

Example of X.509 Certificate (2nd Part)



- ✓ **Tip:** Always confirm the **public key** matches the site you expect, and verify the **CA's signature** to ensure the certificate hasn't been altered.



Public key

Proves the site's identity, used in encryption



CA's signature

Ensures authenticity and integrity

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (2048 bit

Modulus:

00:da:43:c:8:b3:a6:33:5d:83:c0:63:
14:47:fd:6b:22:dd:e84:da:fe:C6:e2:
... (omitted) ...

Signature Algorithm: sha256WithRSAEncryption

4b:a9:64:20:cc:77:0b:30:ab:69:dc:dc:
78:a7f6:e8:14:e9:96:c0:e2:28:1a1:3b:
fb:34:4a:83:e3 ...

Example of X.509 Certificate (3rd Part)



✓ **Tip:** Always check the **extensions** to understand extra attributes and constraints applied to the certificate.



Key Usage

Defines what the public key can be used for (e.g, digital signature, key encipherment)



Extended Key Usage

Specifies additional allowed purposes like server authentication or client authentication



Certificate Policies

States the rules and policies under which the certificate was issued



CRL / OCSP Info

X509v3 extensions:

X509v3 Key Usage: critical

Digital Signature, Key Encipherment

X509v3 Extended Key Usage:

TLS Web Server Authentication,

TLS Web Client Authentication

X509v3 Certificate Policies:

Policy: 2.23.140.1.2.1

X509v3 CRL Distribution Points:

URI: <http://crl3.digicert.com/Example.crl>

Authority Information Access:

OCSP – URI: <http://ocsp.digicert.com>

X.509 Certificates — Standard & Framework for Authentication



Definition

- ITU-T Recommendation X.509 is part of the X.500 series that defines a directory service.
- Provides a framework for authentication services via the X.500 directory.

Core Functions

- Serves as a repository for public-key certificates.
- Supports alternative authentication protocols using public-key certificates.

Technical Note

- The standard does not enforce a specific algorithm but recommends RSA.

Key Facts

- ☐ Widely used for securing websites, email, code signing, and VPN access.
- ☐ Works with certificate authorities to ensure trust and identity verification.

X.509 Overview



Definition

Part of X.500
Directory



Core Functions

Holds public-key
certificates



Technical Note

Recommends
RSA, flexible



Key Facts

Secures web,
email, VPN



Directory
(X.500)



Auth
Frame



Public-Key
Certificates

Foundation of Digital
Certificate Trust

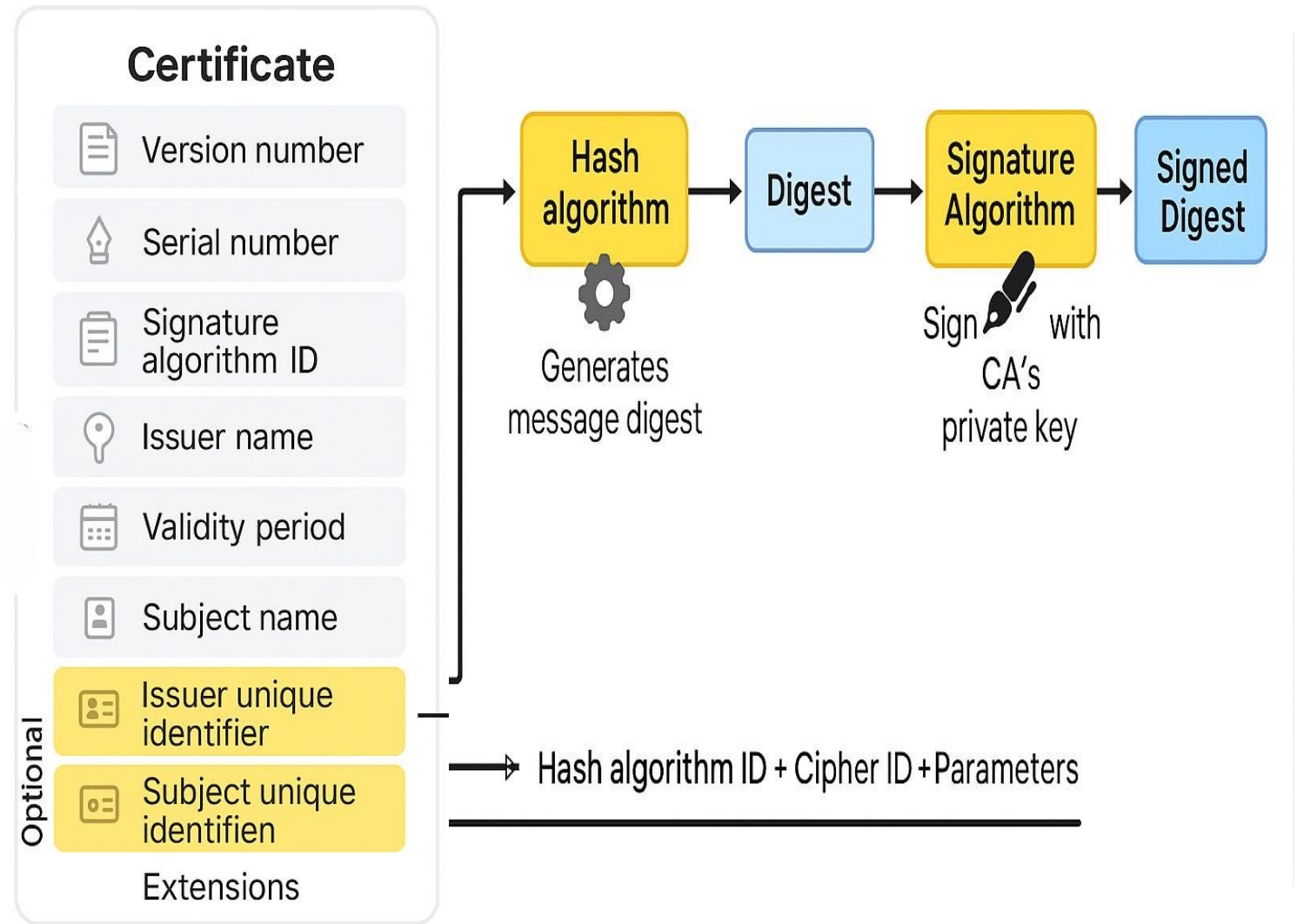
X.509 Functionalities — Structure & Signing Process



- ❑ **Mandatory Fields:** Essential identification and validity details for the certificate.
- ❑ **Optional Fields:** Provide extended identity and flexibility (issuer/subject IDs, extensions).

Signing Process

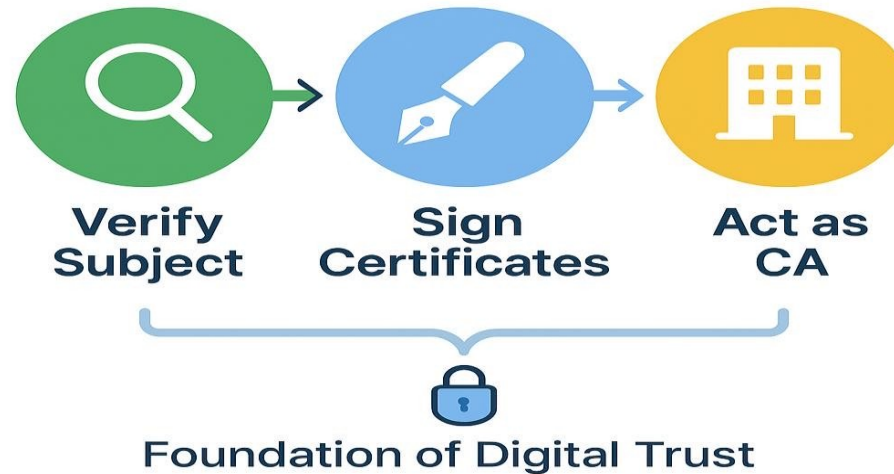
- **Hashing:** Protects data integrity by creating a unique digest.
- **Digital Signature:** CA's private key signs the digest to ensure authenticity.
- **Verification:** Anyone with the CA's public key can validate the signature.



Core Functions of a Certificate Authority (CA)



- Confirms the applicant either owns or legitimately represents the identity in the certificate's **Subject** field.
- **Sign Digital Certificates**
- Generates a **digital signature** using the CA's private key.
- Once applied, the certificate **cannot be modified** without invalidating the signature.
- Anyone can verify authenticity using the **CA's public key**.
- **Operate as a Certificate Authority**
- Issues certificates following verification processes.
- Provides certificates upon application and approval.
- Supports web server setup and secure communication using issued certificates.

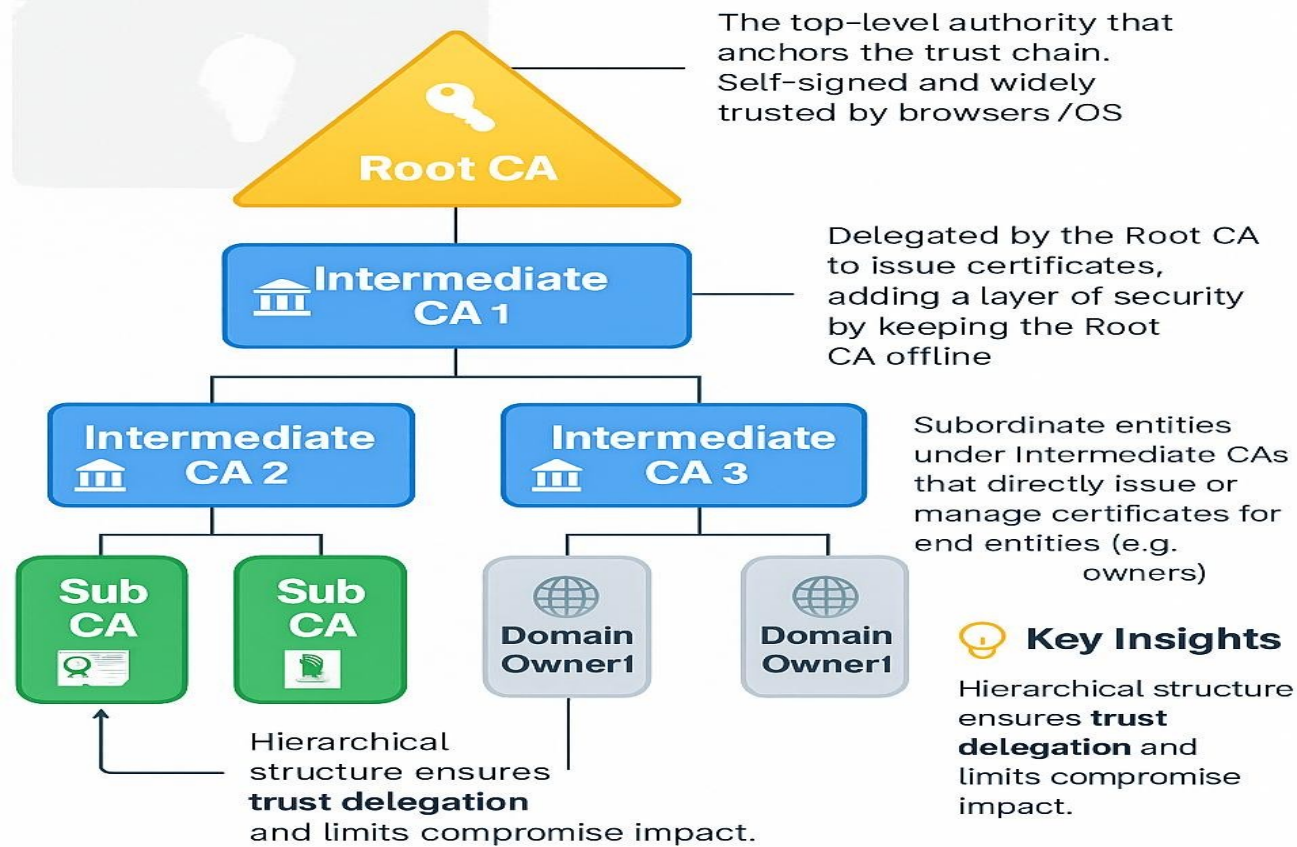


- **Tip:** Always ensure the CA is reputable before trusting a certificate it issues.

Organization of Real World CAs



Root and Intermediate Certificate Authorities



Key Insights

Hierarchical structure ensures **trust delegation** and limits compromise impact.



Key Insights

If an **Intermediate CA** is compromised, Root CA can revoke it without affecting the entire trust ecosystem.

Root CAs and Self-Signed Certificate



Root CA Certificate

- A root CA's public key is stored in an X.509 certificate.
- This certificate is *self-signed*.

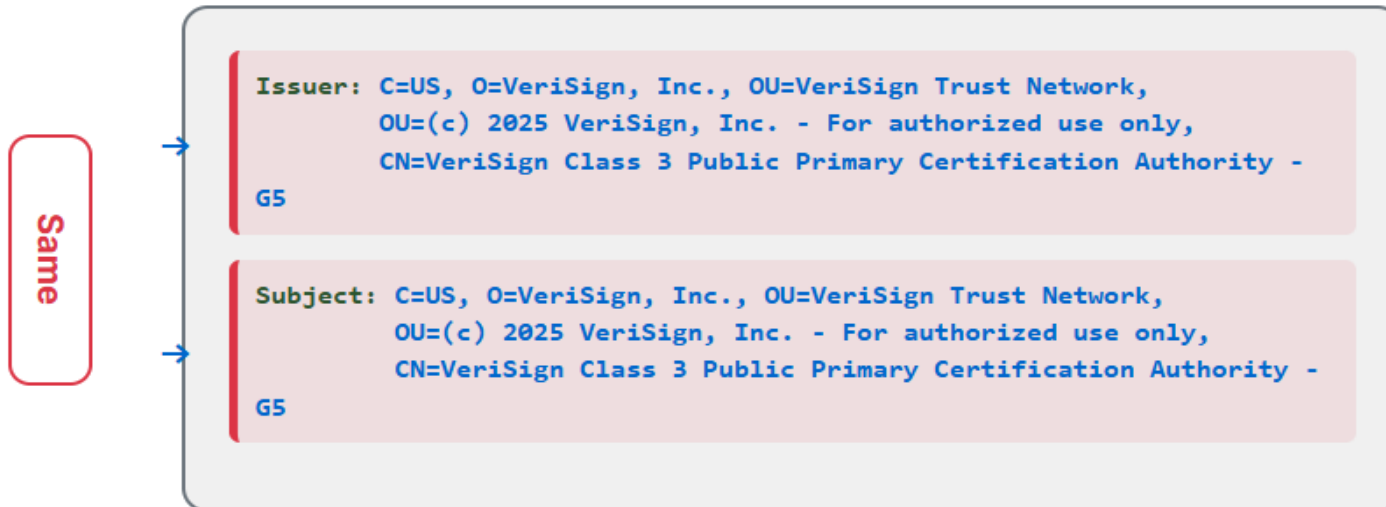
Self-Signed Definition

- The issuer and subject entries in the certificate are **identical**.
- Indicates the CA is at the top of the trust chain.

Trust Mechanism

- Public keys of root CAs are **pre-installed** in operating systems, browsers, and other trusted software.
- This allows devices to automatically verify certificates issued by the root CA.

💡 **Key Takeaway**
Without root CA trust, the entire certificate chain fails.



Step 1

Store public key in X.509 certificate



- A root CA's public key is embedded in an X.509 self-signed certificate

Step 2

Self-signed structure



- Issuer and Subject entries are identical.
- Example:
 - Issuer: VeriSign ...
 - Subject: VeriSign ...

Step 3

Trust establishment



- Public keys of root CAs are pre-installed in:
 - Operating systems
 - Browsers
 - Other software

Trusted by default in major platforms

Intermediate CAs and Chain of Trust



```
$ openssl s_client -showcerts -connect www.paypal.com:443
```

Certificate chain

A is used to
verify B



B



```
0  s: ... /CN=www.paypal.com
   i: ... /CN=Symantec Class 3 EV SSL CA - G3
   -----BEGIN CERTIFICATE-----
   MIHWTCCKGQAwIBAGIQLNQVEPQ3QW5KQSAFavbCfzANBgkqhkiG9w0BAQsFADB3
   ...
   -----END CERTIFICATE-----
```

Intermediate CA's
certificate

```
1  s: ... /CN=Symantec Class 3 EV SSL CA - G3
   i: ... /CN=VeriSign Class 3 Public Primary Certification Authority - G5
   -----BEGIN CERTIFICATE-----
   MIEKzCCBBGQAwIBAGIQfuFKb2/v8tN/P61TTratDANBgkqhkiG9w0BAQsFADCB
   ...
   -----END CERTIFICATE-----
```

PayPal's certificate

Something else is needed to verify A (certificate from another intermediate CA or root CA)

Manually Verifying a Certificate Chain



- **Paypal.pem:** Save Paypal's certificate to a file
- **Symantec-g3.pem:** Save certificate from "Symantec Class 3 EV SSL CA - G3"
- **VeriSign-G5.pem:** Save the VeriSign-G5's certificate from the browser

Root CA's certificate



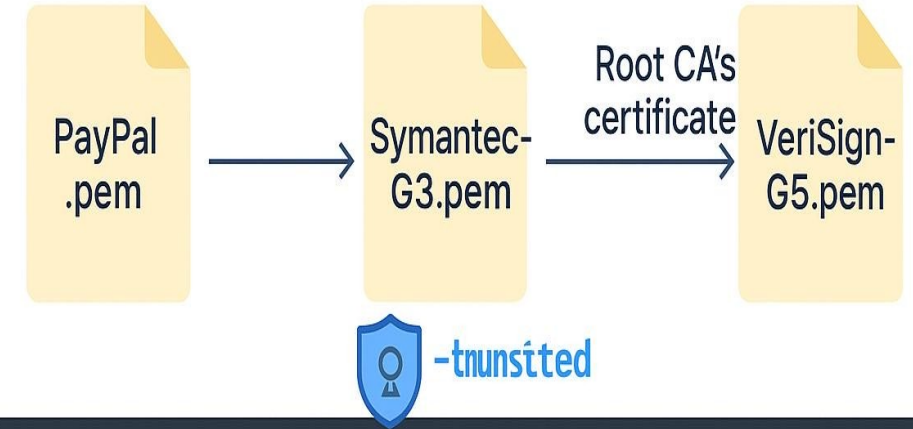
```
$ openssl verify -verbose -CAfile VeriSign-G5.pem  
                        -untrusted Symantec-G3.pem Paypal.pem
```

Paypal.pem: OK



Chain of certificates

Step-by-Step Verification



```
$ openssl verify -verbose -CAfile VeriSign-G5.pem \  
-untrusted Symantec-G3.pem PayPal.pem
```



Chain Verified Successfully

- **Root CA's certificate:** VeriSign-G5.pem is the trusted anchor.
- **Chain of certificates:** Symantec-G3.pem bridges PayPal's certificate to the root CA.

Creating Certificates for Intermediate CA



- ❑ When generating a certificate for an intermediate CA, we need to do something special:

```
$ openssl ca -in modelIntCA.csr -out modelIntCA_cert.pem -md sha256  
-cert modelCA_cert.pem -keyfile modelCA_key.pem  
-extensions v3_ca
```

- ❑ The extension field of the certificate will look as follows:

```
X509v3 extensions:  
→ X509v3 Basic Constraints:  
   CA:TRUE
```

TRUE means the certificate can be used to verify other certificates, i.e, the owner is a CA.
For non-CA certificates, this field is **FALSE**.

- Intermediate CAs act as the “middleman” between the root CA and the end-entity certificate.
- Setting CA:TRUE is essential for the intermediate CA to sign other certificates.

Trusting Certificate



? The Question

If a certificate is **self-signed**, how do we verify it?

🔴 The Reality

There is no way to verify a self-signed certificate.

Instead, trust depends entirely on **how** the certificate was obtained.

❑ Trusted Sources

- 🟡 Comes with the **operating system** → trust OS = trust cert.
- 📦 Comes with the **software** → trust software = trust cert.
- A **Manually added** → trust your own decision.

❑ Untrusted Source

- 📧 Sent by **someone you don't trust** → **do NOT** trust the certificate.

Key Facts

- Self-signed certificates are only as trustworthy as their distribution method.
- Built-in OS and software certificates are usually pre-vetted.
- External, unsolicited certificates pose high security risks.
- ✓ **Action:** Review your lab instructions for certificate handling best practices.

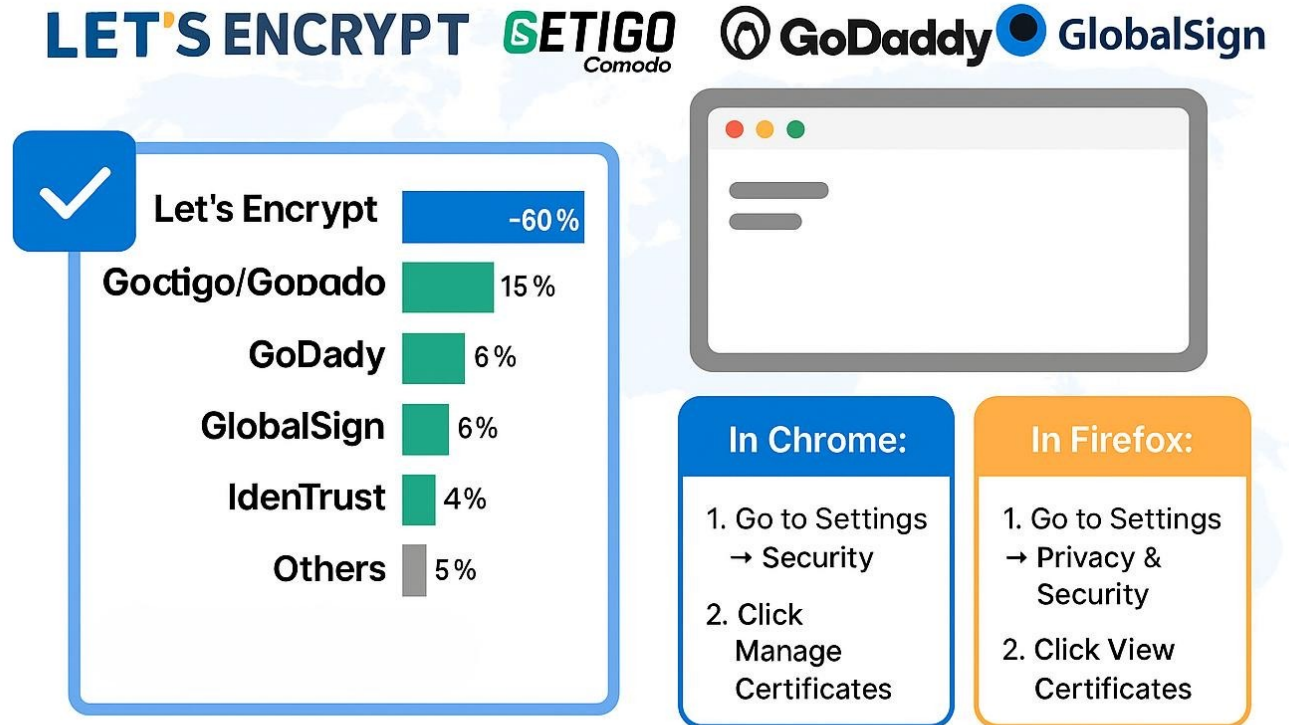


Trusted Certificate Authorities: Market Reality & Browser Access



- Not every trusted CA is included in all browsers — recognition varies by platform.
- According to recent industry statistics ([W3Techs](#), [Sci-Tech-Today](#)):

- ❑ **Let's Encrypt** leads with ~60% of the global SSL/TLS certificate market.
- ❑ **Sectigo/Comodo** holds around 15%.
- ❑ **GoDaddy** and **GlobalSign** each account for about 6%.
- ❑ **IdenTrust** has ~4%.
- ❑ All other providers together make up ~5%.



Trusted by default

How to Check Your Browser's Trusted CAs

▪ Chrome

Open **Settings**, navigate to **Security**, then select **Manage Certificates** to view the list of trusted authorities.

▪ Firefox

Go to **Settings** → **Privacy & Security**, scroll to **Certificates**, and click **View Certificates** to see the full CA list.

Defeats the MITM Attack using PKI



Scenario:

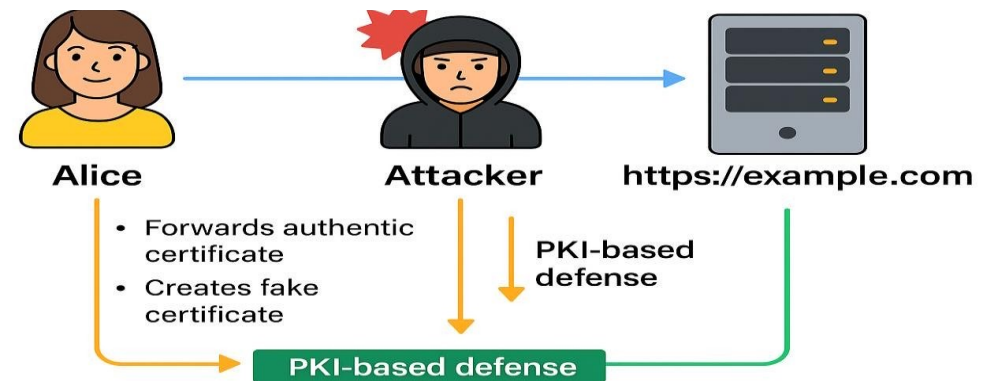
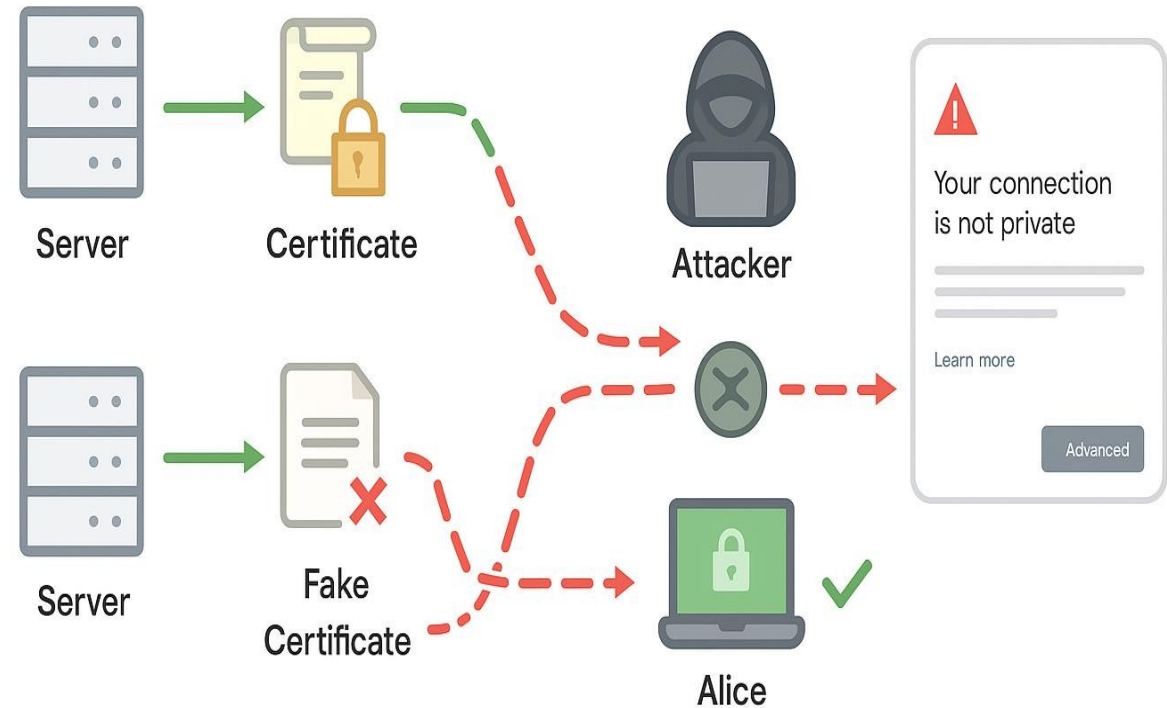
- Alice tries to visit <https://example.com>.
- The server sends its **public key** (in a certificate) to Alice.
- An attacker intercepts the communication.

Possible Attacker Actions:

- Forward the real certificate from example.com.
- Create a fake certificate.
- Send their own certificate to Alice.

How PKI Protects:

- Alice's browser checks the certificate against **trusted Certificate Authorities (CAs)**.
- If the certificate is not signed by a trusted CA, the browser shows a **security warning**.
- PKI ensures the certificate matches the **domain** and has not been tampered with.



Attacker Forwards the Authentic Certificate



1. Attacker's Action

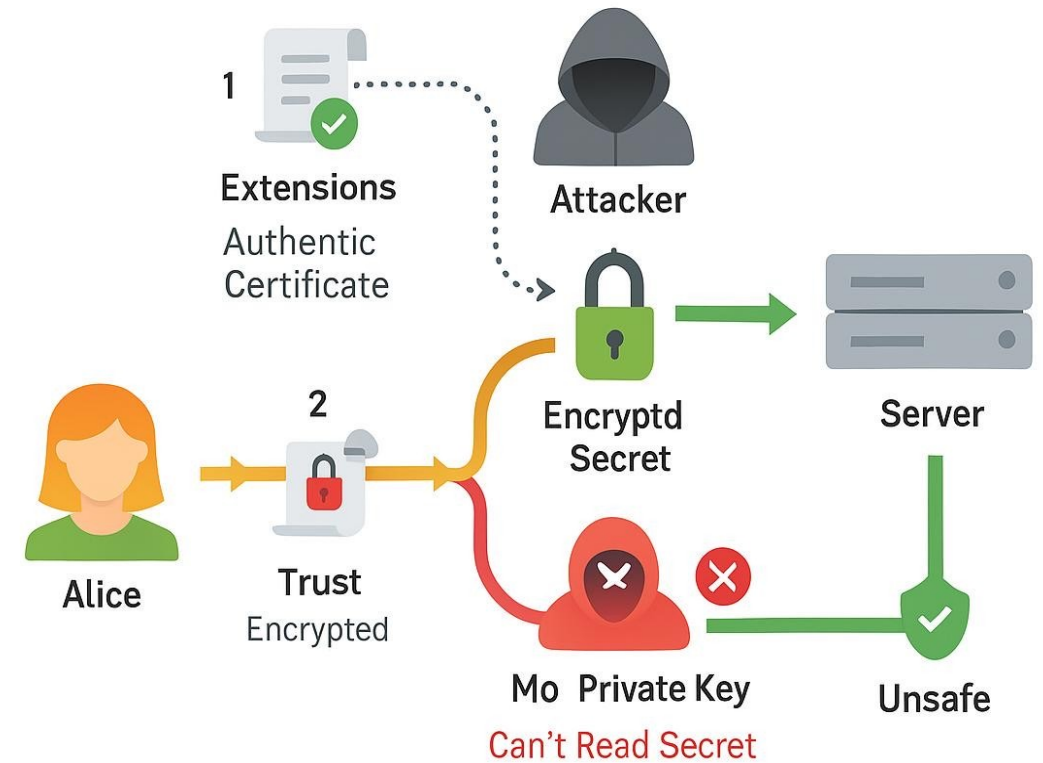
1. Malroy forwards the authentic certificate from the legitimate server.

2. Secure Exchange

1. Alice sends a **secret**, encrypted using the server's public key.
2. This secret is used to establish an encrypted channel between Alice and the server.

3. Why the Attack Fails

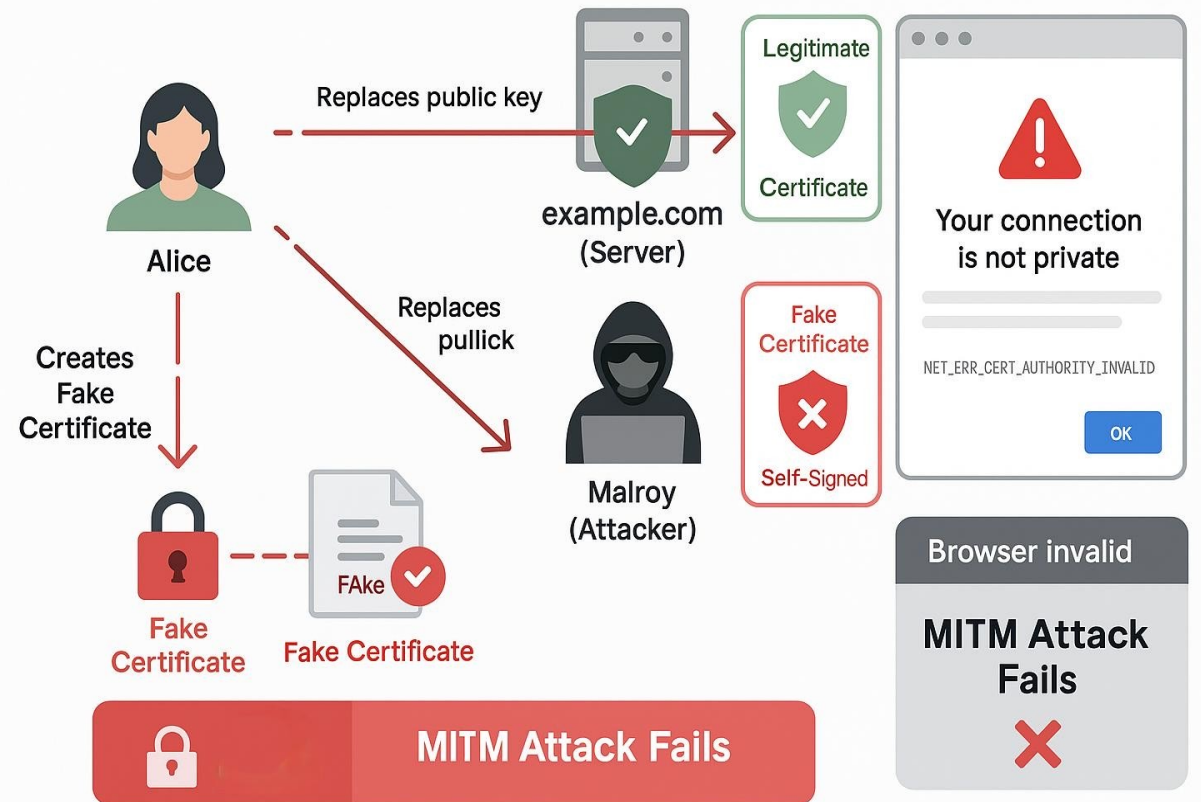
1. Malroy doesn't have the matching private key, so they **cannot decrypt the secret**.
2. Without the secret, Malroy cannot read or modify the communication.
3. Only possible disruption is a **Denial-of-Service (DoS)** attack.



The MITM attack fails because encryption and certificate verification prevent impersonation.

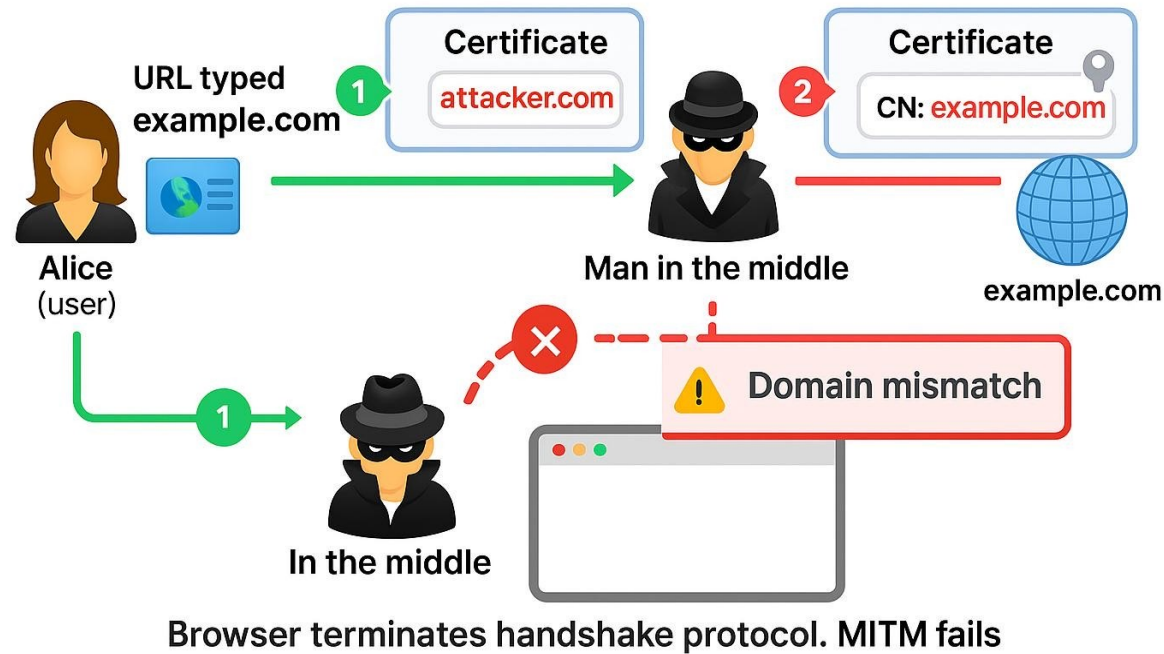
Attacker Creates a Fake Certificate

- Attacker (Malroy) creates a fraudulent certificate for the example.com domain.
- Malroy replaces the server's public key with his own public key.
- Trusted Certificate Authorities (CAs) will not sign Malroy's certificate request because he does not own the domain.
- Malroy can self-sign the fraudulent certificate, creating a **self-signed certificate**.
- Alice's browser cannot find any trusted authority to verify this certificate and displays a **security warning**:



✓ **MITM attack fails** if the user decides to terminate the connection.

Attacker Sends His/Her Own Certificate



- The attacker (Malroy) presents a **valid certificate**, but it is for a different domain (e.g., **attacker.com** instead of **example.com**).
 - The browser verifies if the **certificate's subject field** matches the URL the user intended to visit.
 - **Mismatch Detected:** **attacker.com** $\neq$ **example.com**
 - When the mismatch is found, the browser **terminates the handshake protocol** to prevent the connection.
- ✓ **MITM attack fails** due to **hostname mismatch validation**.

The Man-In-The-Middle Proxy (How it works)



1. Proxy Setup

- Creates a **self-signed CA certificate**.
- The certificate is **installed** in the user's browser as trusted.

2. Traffic Redirection

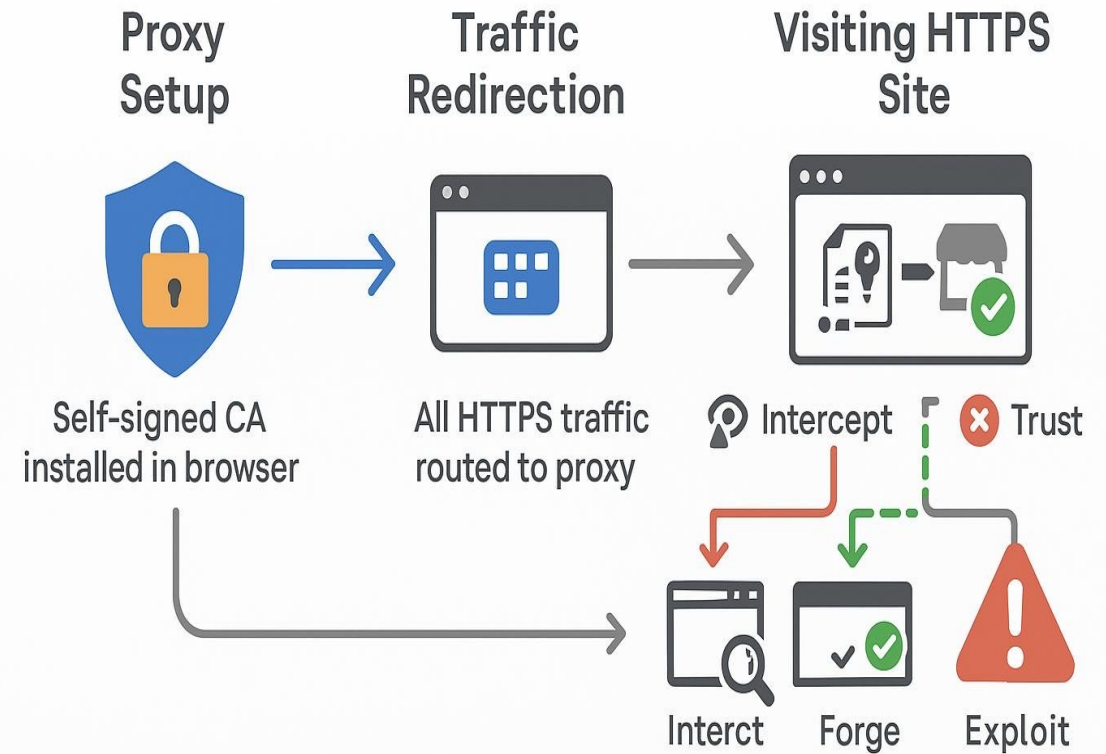
- User machine routing is configured.
- All outgoing **HTTPS traffic** is directed to the proxy machine.

3. When Visiting an HTTPS Site

- **Intercept**: Proxy captures the communication request.
- **Forge**: Proxy generates a **fake certificate** for the target site.
- **Trust Exploit**: Since the browser trusts the proxy's CA, it validates all fake certificates.
- **Outcome**: Proxy successfully becomes a **Man-in-the-Middle**

Key Security Implication:

If a malicious proxy's CA is trusted by a browser, **all HTTPS connections can be intercepted and decrypted** without the user's knowledge.



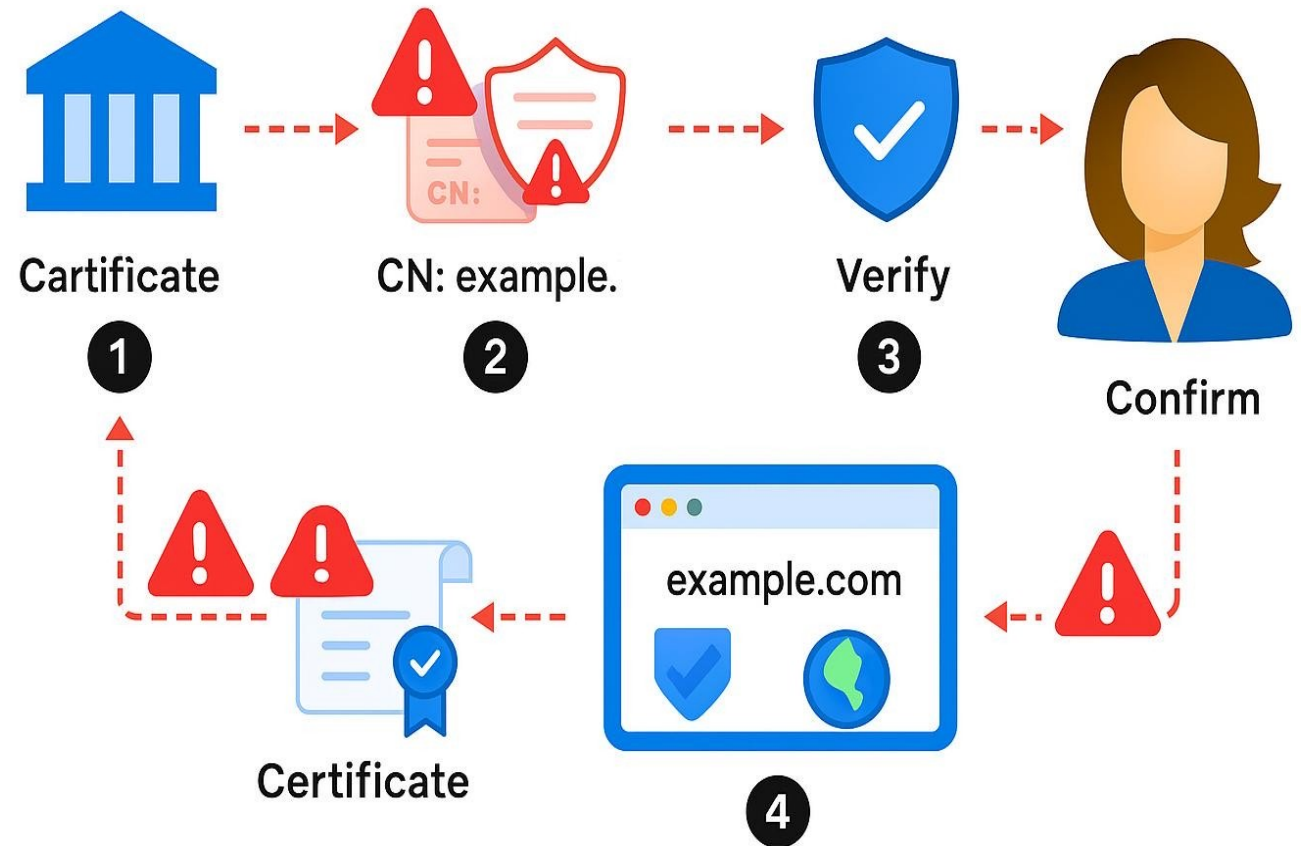
Security Risk: If a malicious proxy's CA is trusted, all HTTPS connections can be intercepted & decrypted without user sees

Attacks Surfaces on PKI



Certificate Authority Approves – The CA issues a certificate for example.com after validating the domain and the requester.

1. **Certificate with CN: example.com** – The issued certificate includes the Common Name (CN) for the site.
2. **Verification against Trusted Certificates** – Alice's browser compares the received certificate with its preloaded list of trusted CA certificates.
3. **User Confirmation** – Once verified, Alice's browser confirms the connection as secure.



Key Point:

If any part of this chain is compromised (fake CA, manipulated CN, or altered trust list), the system becomes vulnerable to attacks.

Real-life Attack on CA's



Incident	Date	Impact	Source
DigiCert DCV Failure	July 2024	83,000+ certificates revoked due to domain validation flaw	Daily CyberSecurity
Entrust Distrust	Nov 2024	Browsers block new certs from Entrust roots due to repeated compliance issues	CyberMaterial
Let's Encrypt Renewal Failure	Apr 2024	Geoblocked domains failed renewal after CT perspective change	community.letsencrypt.org
GoDaddy SCT Logging Incident	May–June 2025	Thousands of Safari-failing certs due to CT log misconfig	sslmate.com



Post-Quantum Cryptography



Post-Quantum Cryptography (PQC) refers to cryptographic algorithms—usually public-key algorithms—designed to be secure against attacks from both classical and future **quantum computers**.

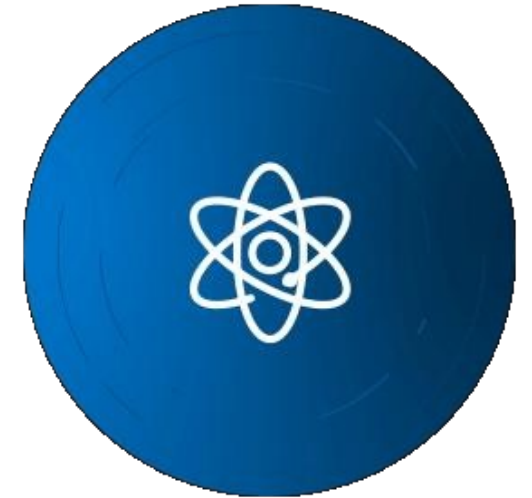
Why Do We Need PQC Now?

"Harvest Now, Decrypt Later" (HNDL): Attackers are recording encrypted data today, intending to decrypt it once a powerful quantum computer exists.

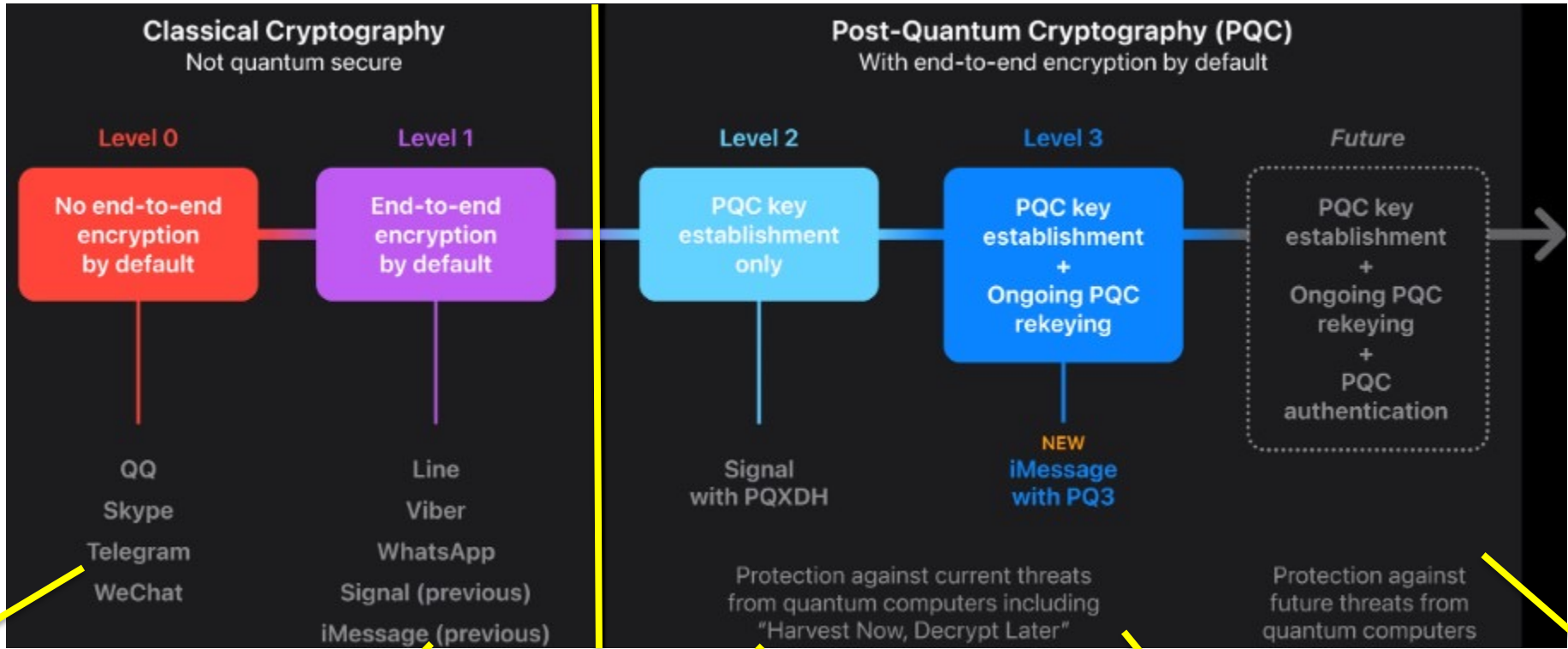
The "Winners" (NIST Standards 2024)

In August 2024, NIST released the first finalized standards that the world is now adopting:

- **ML-KEM (FIPS 203):** The primary standard for general encryption (derived from CRYSTALS-Kyber).
- **ML-DSA (FIPS 204):** The primary standard for digital signatures (derived from CRYSTALS-Dilithium).
- **SLH-DSA (FIPS 205):** A backup hash-based signature standard (derived from SPHINCS+).



iMessage with PQ3: The new state of the art in quantum-secure messaging at scale



Your messages are not fully private; the service provider could read them

Messages are protected so only you and the receiver can read them but still not safe against quantum computers.

Uses quantum-safe algorithms to set up the encryption key, but not for ongoing rekeying. Protects against "Harvest Now, Decrypt Later" attacks

Uses PQC both to set up and continuously refresh encryption keys, giving stronger protection against future quantum attack

I'm really talking to Bob not an impersonator pretending to be Bob

Zero Knowledge Proofs



A Zero-Knowledge Proof (ZKP) is a cryptographic method allowing a "prover" to convince a "verifier" that a statement is true without revealing any information beyond the validity of that statement.

- **Origin:** The concept was introduced in the 1985 paper "*The Knowledge Complexity of Interactive Proof-Systems*".
- **Inventors:** The paper was authored by **Shafi Goldwasser**, **Silvio Micali**, and **Charles Rackoff** at MIT.
- **Recognition:** The researchers received the 1993 Gödel Prize for this work.



Properties-Zero Knowledge Proofs



Completeness

- If statement is true, **verifier** will be convinced by prover

Soundness

- If statement is false, a **cheating prover** cannot convince verifier it is true
 - Except with some small probability

Zero-Knowledge

- Verifier learns nothing beyond the statement's validity

Property : Completeness

(Prover)



You

Information is true.

(Verifier)



Friend

Property : Soundness

(Prover)



You

Information is false.



(Verifier)



Friend

Property : Zero-knowledge

(Prover)



You

Information is true.

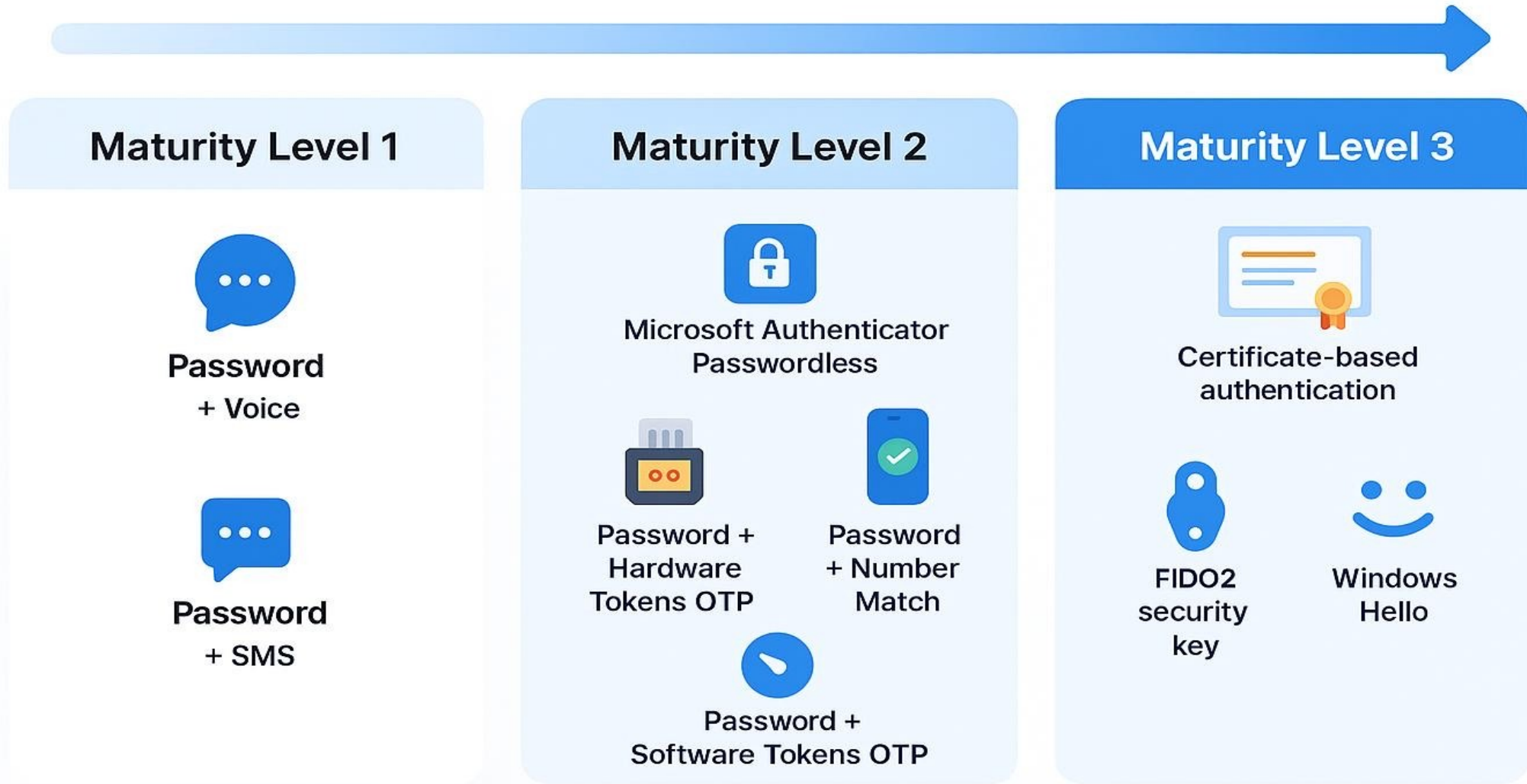


(Verifier)



Friend

User Authentication – Maturity Levels



Higher maturity levels offer stronger protection against credential theft.

Summary



- Public Key Cryptography is the foundation for secure communication
- Keys need infrastructure to identify holders, allow revoking
- Digital signatures provide integrity of origin and content
- Authentication technique prevents unauthorized access
- Use public key cryptosystems help establish Trust



Questions Answers